# Barangay Minante 1 Integrated Management System

Complete Capstone Research and Technical System Documentation

Development of an Integrated Barangay Permit, Event Approval, Blotter Management System with Notification and Scheduling Features for Barangay Minante 1, Cauayan City

Source-grounded local implementation edition

October 7, 2026

# Document Information

Purpose: capstone research, defense preparation, user training, technical handover and maintenance.

Prepared from the local source and read-only SQLite schema. No institutional approval, author identity or sign-off is asserted. The project team should complete its required institutional cover/approval forms separately.

Evidence: prior October 5 audit plus the current captured acceptance run. Current capabilities and partial workflows are distinguished throughout. Repository identity is supplied by the project owner; no remote commit is certified.

Editable counterpart: Barangay_Minante_1_Integrated_Management_System_Full_Documentation.docx. Narrative source, schema evidence, generation scripts and validation summary are included in documentation/.

# Table of Contents

**Executive Summary** 10

**Chapter 1 — System Introduction** 11

1.1 Background and problem context . . . 11
1.2 Purpose of the system . . . 11
1.3 General objective . . . 11
1.4 Specific objectives . . . 11
1.5 Scope . . . 12
1.6 Significance . . . 12
1.7 Research and documentation boundaries . . . 12

**Chapter 2 — System Overview** 13

2.1 Users and role separation . . . 13
2.2 Permission matrix . . . 13
2.3 Module map and implementation classification . . . 16
2.4 Input–Process–Output model . . . 16
2.5 Data flow diagram . . . 17

**Chapter 3 — Architecture and Technology** 19

3.1 Architecture and layers . . . 19
3.2 Technologies actually used . . . 20
3.3 Real project structure . . . 21
3.4 Frontend behavior . . . 21

**Chapter 4 — Complete Feature and Functionality Documentation** 23

4.1 Registration, account management and verification . . . 23
4.2 Resident Type versus Residency Status . . . 24
4.3 Login, password recovery and profile . . . 24
4.4 Permit processing and dynamic requirements . . . 24
4.5 Pricing, electronic signature and approved document . . . 26
4.6 Event approval and reservation conflict logic . . . 26
4.7 Blotter management . . . 27
4.8 Walk-in assistance, search and acknowledgments . . . 27
4.9 Queue, priority and release counter . . . 29
4.10 Appointments and assignment . . . 29
4.11 Payment management . . . 30
4.12 General concerns, referrals and resident updates . . . 30
4.13 Household management . . . 31

4.14 Scholarship and Ayuda programs . . . 31
4.15 Announcements and timed visibility . . . 32
4.16 Notifications, activity logs and feedback . . . 32
4.17 Role dashboards and monitoring . . . 33

**Chapter 5 — SQLite Database Documentation 34**

5.1 Database purpose and connection . . . 34
5.2 Primary keys, foreign keys and relationships . . . 34
5.3 Simplified entity relationship diagrams . . . 34
5.4 Database table catalog . . . 37
5.5 Constraints and integrity . . . 38
5.6 CRUD with actual examples . . . 38
5.7 Transactions, commit and rollback . . . 39
5.8 Queries, indexes and persistence . . . 39
5.9 Initialization and additive migration . . . 39

**Chapter 6 — Code and Logic Reference 40**

6.1 How to follow a request through the source . . . 40
6.2 Important functions/classes . . . 40
6.3 Selected short code examples . . . 43
6.4 State transitions and repeated actions . . . 43
6.5 Compatibility names and inactive fields . . . 43

**Chapter 7 — Security and Privacy 44**

7.1 Implemented controls . . . 44
7.2 Privacy access boundaries . . . 44
7.3 Security and privacy limits . . . 45
7.4 Error handling . . . 45

**Chapter 8 — Reports and Analytics 46**

8.1 Real-data reporting pipeline . . . 46
8.2 Implemented reporting areas . . . 46
8.3 Visualization choices . . . 47
8.4 Word export and privacy . . . 47

**Chapter 9 — Testing and Quality Assurance 48**

9.1 Evidence and methodology . . . 48
9.2 Test cases and actual results . . . 49
9.3 Performance observations . . . 51
9.4 Failed, partial and untested areas . . . 51

**Chapter 10 — User Manuals** **52**

10.1 Resident manual . . . 52
10.2 Barangay Staff manual . . . 52
10.3 Administrator manual . . . 53

**Chapter 11 — Deployment, Maintenance and Backup** **55**

11.1 Environment and dependencies . . . 55
11.2 Configuration reference . . . 55
11.3 Deployment boundary . . . 56
11.4 SQLite backup and recovery . . . 56
11.5 Maintenance and handover . . . 56
11.6 GitHub and documentation maintenance . . . 56

**Chapter 12 — Strengths, Limitations and Future Work** **58**

12.1 Verified strengths . . . 58
12.2 Current limitations and weak areas . . . 58
12.3 Future enhancements — not current features . . . 59

**Chapter 13 — Capstone Defense Reviewer** **60**

Purpose and research . . . 60
Users and permissions . . . 60
Accounts and security . . . 61
Database and programming . . . 61
Permits and events . . . 62
Front desk and payments . . . 63
Programs and announcements . . . 64
Reports, testing and limits . . . 64

**Glossary** **66**

**Appendix A — Complete SQLite Data Dictionary** **68**

A.1 accepted_id_types . . . 68
A.2 activity_logs . . . 68
A.3 announcements . . . 69
A.4 application_requirements . . . 69
A.5 assistance_referrals . . . 70
A.6 blotter_cases . . . 71
A.7 community_programs . . . 72
A.8 event_categories . . . 73
A.9 event_requests . . . 74
A.10 event_reservations . . . 75

A.11 event_venues . . . 76
A.12 general_concerns . . . 77
A.13 household_members . . . 77
A.14 households . . . 78
A.15 notifications . . . 78
A.16 password_reset_tokens . . . 79
A.17 payment_authorities . . . 80
A.18 payments . . . 80
A.19 permit_applications . . . 81
A.20 permit_signatories . . . 83
A.21 permit_types . . . 83
A.22 program_application_history . . . 84
A.23 program_applications . . . 84
A.24 queue_entries . . . 85
A.25 reports . . . 86
A.26 request_assignments . . . 87
A.27 requirement_history . . . 87
A.28 resident_feedback . . . 88
A.29 resident_profiles . . . 88
A.30 resident_types . . . 90
A.31 resident_update_requests . . . 90
A.32 schedules . . . 91
A.33 service_requirements . . . 92
A.34 system_settings . . . 92
A.35 users . . . 93

**Appendix B — Route Inventory** **94**

**Appendix C — Source Evidence and Reproduction** **99**

**Appendix D — Final Documentation Verification** **103**

# List of Figures

Figure 1. Input–Process–Output model

Figure 2. Seven core processes and their data stores

Figure 3. Implemented application architecture

Figure 4. Administrator dashboard from the synthetic browser audit

Figure 5. Registration and account-access workflow

Figure 6. Permit review, payment, signing and release workflow

Figure 7. Event request and private recommendation workflow

Figure 8. Implemented blotter state progression

Figure 9. Walk-in intake and queue workflow

Figure 10. Document release checks

Figure 11. Payment or waiver decision workflow

Figure 12. Community program and application workflow

Figure 13. Announcement lifecycle and preservation

Figure 14. Resident, identity and household relationships

Figure 15. Permit, requirements, queue, signature and payment relationships

Figure 16. Events, blotters and shared operational records

Figure 17. Programs, applications, history and announcements

# List of Tables

Table 1. 1.5 Scope

Table 2. 2.2 Permission matrix

Table 3. 2.3 Module map and implementation classification

Table 4. 3.1 Architecture and layers

Table 5. 3.2 Technologies actually used

Table 6. 4.8 Walk-in assistance, search and acknowledgments

Table 7. 5.4 Database table catalog

Table 8. 5.6 CRUD with actual examples

Table 9. 7.2 Privacy access boundaries

Table 10. 7.4 Error handling

Table 11. 8.2 Implemented reporting areas

Table 12. 9.2 Test cases and actual results

Table 13. 11.2 Configuration reference

Table 14. 12.2 Current limitations and weak areas

Table 15. Glossary

Table 16. A.1 accepted_id_types

Table 17. A.2 activity_logs

Table 18. A.3 announcements

Table 19. A.4 application_requirements

Table 20. A.5 assistance_referrals

Table 21. A.6 blotter_cases

Table 22. A.7 community_programs

Table 23. A.8 event_categories

Table 24. A.9 event_requests

Table 25. A.10 event_reservations

Table 26. A.11 event_venues

Table 27. A.12 general_concerns

Table 28. A.13 household_members

Table 29. A.14 households

Table 30. A.15 notifications

Table 31. A.16 password_reset_tokens

Table 32. A.17 payment_authorities

Table 33. A.18 payments

Table 34. A.19 permit_applications

Table 35. A.20 permit_signatories

Table 36. A.21 permit_types

Table 37. A.22 program_application_history

Table 38. A.23 program_applications

Table 39. A.24 queue_entries

Table 40. A.25 reports

Table 41. A.26 request_assignments

Table 42. A.27 requirement_history

Table 43. A.28 resident_feedback

Table 44. A.29 resident_profiles

Table 45. A.30 resident_types

Table 46. A.31 resident_update_requests

Table 47. A.32 schedules

Table 48. A.33 service_requirements

Table 49. A.34 system_settings

Table 50. A.35 users

Table 51. Appendix B — Route Inventory

Table 52. Appendix C — Source Evidence and Reproduction

# Executive Summary

The Barangay Minante 1 Integrated Management System is a browser-based application for recording and managing resident accounts, permit and certificate requests, event requests, blotter cases, appointments, walk-in assistance, payments, community programs, announcements, and operational reports. Its full project title is **Development of an Integrated Barangay Permit, Event Approval, Blotter Management System with Notification and Scheduling Features for Barangay Minante 1, Cauayan City**.

Residents submit and monitor their own transactions. Barangay Staff assist residents, verify requirements, and endorse requests. Administrators manage configuration, approve accounts and transactions, authorize signatures and financial personnel, and review system-wide reports. This division supports accountability while allowing Staff to submit personal resident transactions without processing their own requests.

The implementation uses Python, Flask, Flask-SQLAlchemy, SQLite, Jinja templates, custom HTML/CSS/JavaScript, python-docx, and matplotlib. Data persists in a SQLite file; private evidence and signature images are stored outside the public static directory. The current application produces editable DOCX acknowledgments, approved documents, payment receipts, and reports. This documentation package additionally uses ReportLab to produce its PDF; ReportLab is not an application export feature.

The documentation was prepared from the local final working tree and a read-only schema inspection, covering 35 SQLite tables, 72 declared application routes plus Flask's static route, and the available automated and browser audit evidence. The current captured acceptance run passed **48 tests in 84.587 seconds (OK)**. The saved browser audit covers 160 page checks. These results establish behavior within the exercised scenarios, not proof of complete security or absence of defects.

Several boundaries matter for defense and operation: SMS and email delivery are simulated; payment recording is manual cash or an Administrator waiver; signatures are electronic images rather than cryptographic signatures; appointment rescheduling changes status without editing the scheduled time; and automatic priority ordering and complete referral follow-up are not implemented. Address and ID edits restart verification in the current code. The following chapters distinguish implemented behavior, partial workflows, and future work.

# Chapter 1 — System Introduction

## 1.1 Background and problem context

Barangay transactions involve personal records, supporting evidence, decisions, dates, and communication between residents and personnel. Paper-based or disconnected recording methods can make it difficult to find previous submissions, follow a request's status, detect overlapping venue use, or prepare consolidated reports. These are the kinds of problems the developed system is designed to address. This document does not claim that an interview, survey, time study, or measured baseline established their frequency in Barangay Minante 1.

The project brings related transactions into one application. Instead of interpreting a submitted form as approval, it records separate submission, verification, endorsement, decision, payment, signature, and release stages. This distinction is central to the design: each stage has its own meaning and authorized actor.

## 1.2 Purpose of the system

The purpose is to centralize resident and service records, support traceable processing, and provide information for authorized monitoring. Core services cover permits, events, and blotters. Supporting services include schedules, in-app notifications, front-desk assistance, requirements verification, queue tracking, release recording, household membership, manual payments, Scholarship and Ayuda applications, announcements, feedback, and aggregate reports.

The software supports administrative decisions; it does not determine legal entitlement, independently authenticate an ID, automatically award assistance, or prove that a physical payment or release occurred. Personnel remain responsible for the truth of recorded actions and compliance with local procedures.

## 1.3 General objective

To develop an integrated Barangay management system that digitizes and centralizes resident service processing, monitoring, scheduling, communication records, and administrative reporting for Barangay Minante 1, Cauayan City.

## 1.4 Specific objectives

1. Provide registration, identity/address review, account approval, and role-controlled access.

2. Record permit submissions with configurable requirements and fee snapshots, supporting review through release.

3. Manage event requests and prevent overlapping reservations for the same configured venue.

4. Record complainant/respondent information and track blotter processing states.

5. Support transaction-linked appointments and in-app notifications, with clearly identified simulated email/SMS records.

6. Assist walk-in residents using the same permit workflow, with a queue ticket and acknowledgment.

7. Record authorized payments and waivers separately from configured prices.

8. Manage configurable Scholarship/Ayuda opportunities, applications, claim rules, review, and completion.

9. Publish and expire announcements according to stored dates while preserving history.

10. Generate SQLite-based analytics, editable Word reports, and activity records for authorized oversight.

## 1.5 Scope

**Table 1. 1.5 Scope**

| Area | Current scope | Boundary |
|---|---|---|
| Resident services | Registration, profile, own permits/events/blotters, schedules, notifications, program applications, documents and feedback | Approval gates access; operational records belonging to others are not resident services |
| Staff operations | Read-only user list, front desk, evidence review, endorsement, scheduling, routing and release | No general final-approval, configuration or system-report authority |
| Administration | Account decisions, catalogs, fees, signatories, final decisions, programs, announcements, reports and audit view | Actions still use route-specific validation; a configured signatory must match the acting Administrator |
| Monitoring | 17 aggregate report types, charts, filters and DOCX exports | Not a public data portal, accounting ledger, or staff-ranking system |
| Community programs | Scholarship/Ayuda drafts, publication, applications, review and completion | Human eligibility review; no automatic benefit disbursement |
| Data management | SQLite records, additive migration, private files and database backup utility | Single-service architecture; uploaded files need coordinated backup |

## 1.6 Significance

For Residents, the system provides a reference number, status visibility, evidence submission, and an assisted route for those who need help. For Staff, it organizes intake, review and front-desk records. For the Administrator, it separates final decisions from preliminary review and supplies aggregate monitoring. For Barangay management, it creates a common record base for coordination. For researchers and future developers, the source, schema, test suite, and this document provide a reproducible technical reference. Community benefit is an intended outcome; no satisfaction survey or measured service-time reduction is asserted.

## 1.7 Research and documentation boundaries

This is a source-grounded system description and technical evaluation, not a fabricated empirical research study. No survey responses, respondent counts, acceptance scores, institutional approval signatures, or external research references are invented. The document-information page leaves authorship and institutional approval to the actual project team. Current limitations and future improvements are separated in Chapter 12.

# Chapter 2 — System Overview

## 2.1 Users and role separation

**Resident:** an approved active account holder who manages personal information and submits or monitors their own transactions. A newly registered account remains Pending until the Administrator verifies its ID and address and approves it.

**Barangay Staff:** an operational user who assists residents, examines evidence, endorses transactions and handles front-desk work. Staff can make personal permit/event/blotter submissions using the same account. Backend checks prohibit Staff from processing their own requests; being assigned a request does not override that restriction.

**Administrator:** the role with broad permission grants in `permissions.py`. Administrators make final decisions, configure services, and access reports and activity logs. Additional checks still apply to particular actions: signatories must be configured, payments cannot be self-recorded, and program applications cannot be self-processed.

## 2.2 Permission matrix

The matrix combines `ROLE_PERMISSIONS` with checks inside the actual routes. A sidebar entry alone is not an authorization rule. “Own” means a record owned by the signed-in user.

**Table 2. 2.2 Permission matrix**

| Function | Resident | Staff | Administrator |
|---|---|---|---|
| Public registration | Creates Resident account | No public Staff signup | No public Admin signup |
| Login/profile | Active approved own account | Active authorized own account | Active own account |
| View users | No | Read-only list | Manage/review |
| View uploaded valid ID | Own | Own, not others | Authorized review |
| Verify ID/address, approve registration | No | No | Pending Resident/Staff verification; Resident registration approval |
| Change role/account activity | No | No | Yes, with last-Admin safeguards |
| Create personal permits/events/blotters | Yes | Yes | Routes allow broad permissions; not normal resident role |
| View transaction records | Own | Operational access | Operational access |
| Review/endorse permits/events | No | Yes, except own | Yes |
| Final permit/event approval | No | No | Yes |
| Review permit evidence files | No dedicated owner download route | Yes, except own | Yes |
| Resubmit permit requirements | Own eligible request | Own eligible request | Not ordinary resident resubmission |
| Configure requirements/fees/venues | No | No | Yes |
| Apply electronic signature | No | No | Designated Admin for that service |
| Front desk/queue/households/concerns/referrals | No | Yes | Yes |
| Record permit release | No | Yes, except own | Yes, validated ready/signed state |
| Request appointment | Own transaction | Operational transactions | Operational transactions |
| Confirm/complete appointment | No | Yes | Yes |
| Cancel appointment | No | No | Yes, permitted states |
| Record payment | No | Only explicit payment authority, except own | Yes, except own |
| Waive fee | No | No | Yes, reason required |

| Function | Resident | Staff | Administrator |
|---|---|---|---|
| Download payment receipt | Own Paid record | Own or payment-authorized access | Yes, Paid record |
| Draft programs/announcements | No | Yes | Yes |
| Publish/close program; approve announcement | No | No | Yes |
| Program application | Own approved profile | Assisted approved Resident/Staff | Assisted approved Resident/Staff |
| Program review/qualification | No | Yes, except own | Yes, except own |
| Approve/waitlist/complete program application | No | No | Yes, except own |
| Read/mark notifications | Own | Own | Own |
| Send notification | No | Yes | Yes |
| Reports/charts/Word report export | No | No | Yes |
| Activity Logs/configuration | No | No | Yes |
| Feedback | Own completed permit, once | Own completed permit, once | Route requires ownership |

## 2.3 Module map and implementation classification

**Table 3. 2.3 Module map and implementation classification**

| Module | Classification | What is present |
|---|---|---|
| User/account management | Implemented with correction limits | Registration, verification, approval, roles, profile, reset-token logic |
| Permit processing | Implemented | Snapshots, review, endorsement, decision, payment gate, signature and release |
| Event approval | Implemented | Configured category/venue, conflict prevention and temporary recommendations |
| Blotter management | Partially Implemented | Intake and status progression; stored resolution/officer fields lack a complete editing workflow |
| Scheduling | Partially Implemented | Linked requests and status changes; no appointment time-edit or officer/office assignment workflow |
| Notifications | Partially Implemented | In-app read state works; email/SMS delivery simulated |
| Front desk and queue | Implemented with limits | Assisted permits, ticket state progression and acknowledgment; no automated priority ordering |
| Payments | Implemented within manual scope | Exact cash amount, Admin waiver, receipt and authorization |
| Community programs | Implemented | Scholarship/Ayuda configuration, claims, review, capacity and completion |
| Announcements | Implemented | Draft approval, time-filtered publication/expiry and archival |
| Reports/activity logging | Implemented | SQL aggregates, DOCX, action records and admin view |
| Concerns/referrals | Partially Implemented | Concern intake/routing; referral intake; no full resolution/distribution lifecycle |
| Household management | Partially Implemented | Create, head/member association, transfer and statistics; no dedicated move-out operation |
| Profile update requests | Partially Implemented | Staff-assisted change proposal and Admin decision; direct profile edits coexist |

## 2.4 Input–Process–Output model

**Simple explanation.** Inputs are the information and evidence people provide. Processing validates and reviews that information. Outputs are the stored decisions and documents users receive.

**Technical explanation.** Forms carry a session CSRF token to Flask. Routes validate ownership, permission, fields and workflow state. SQLAlchemy writes related records in a transaction. Jinja pages, notifications, editable documents and aggregates expose the resulting state to authorized users.

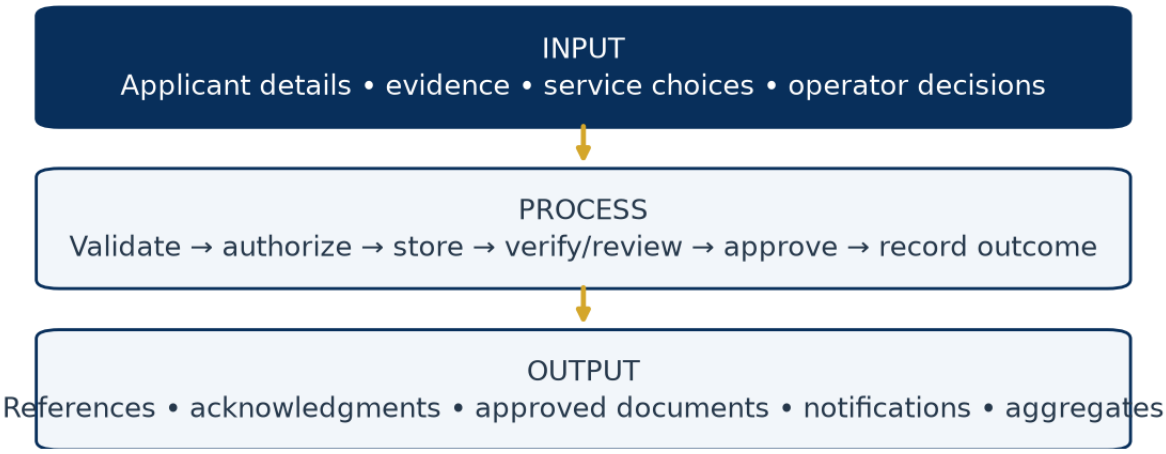



**Figure 1. Input–Process–Output model**

The figure separates applicant evidence from staff decisions and resulting documents. An acknowledgment is an output of intake, not a decision of approval.

## 2.5 Data flow diagram

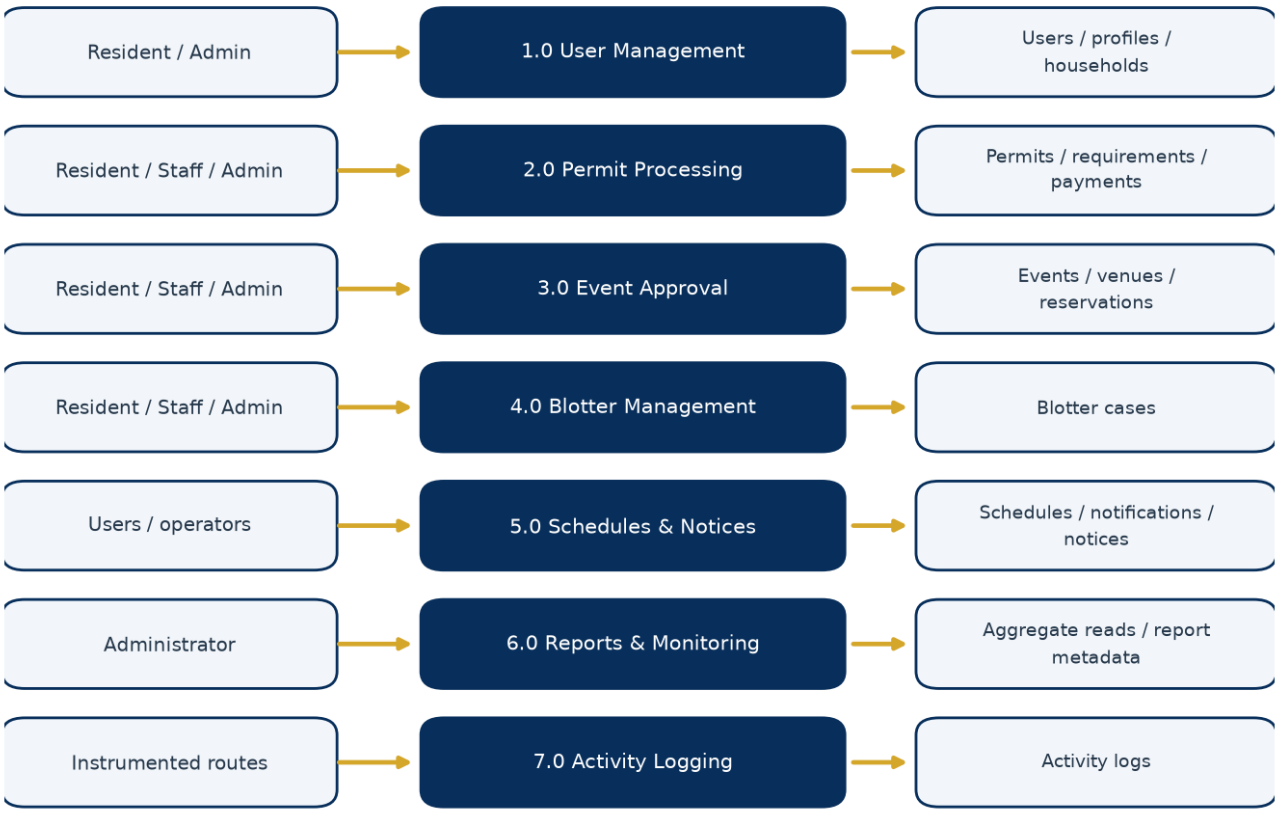


**Figure 2. Seven core processes and their data stores**

This logical DFD groups implemented responsibilities, not separate deployed servers. Process 1.0 manages users/profiles/households; 2.0 manages permits/requirements/signatures/payments; 3.0 manages events/reservations; 4.0 stores blotter cases; 5.0 handles schedules/notifications and announcement visibility; 6.0 reads aggregate data and exports reports; 7.0 records and displays audit actions. Programs extend resident services and monitoring using separate program/application tables. Front-desk intake feeds the same permit records as online requests. A payment receipt is derived from a real Payment row, not the permit-type fee catalog.

# Chapter 3 — Architecture and Technology

## 3.1 Architecture and layers

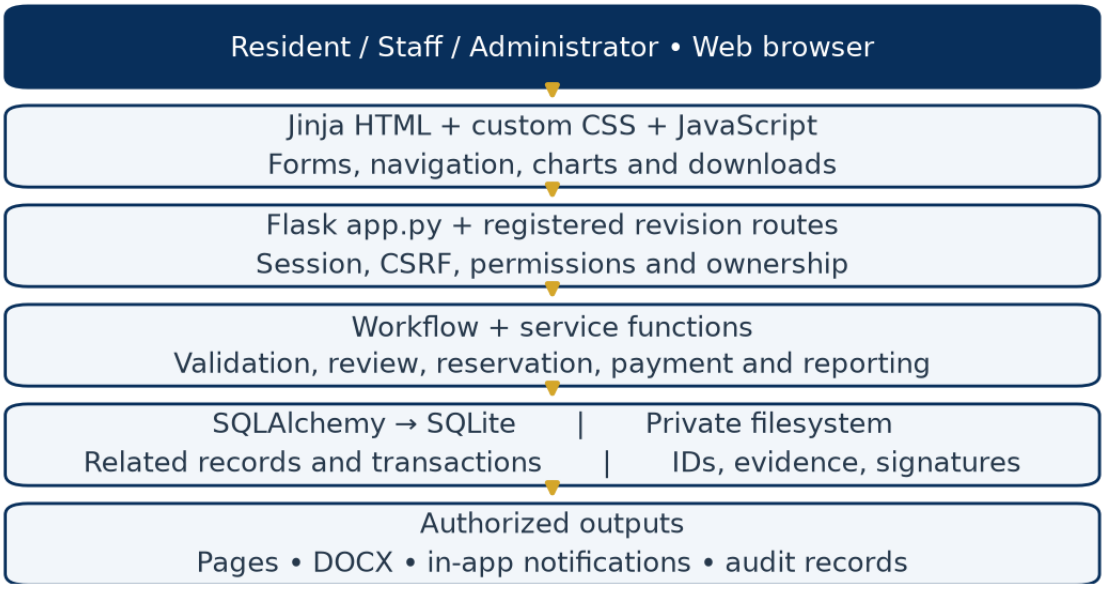



**Figure 3. Implemented application architecture**

**Simple explanation.** The browser displays forms; Flask checks what a person may do; the database remembers the transaction; document and chart services turn authorized records into useful output.

**Technical explanation.** This is a server-rendered Flask monolith, not a microservice or single-page-application architecture. `app.py` owns the Flask instance and most routes. `register_revision_routes()` attaches payment/program routes directly to that instance. There are no Flask Blueprints in the inspected implementation.

**Table 4. 3.1 Architecture and layers**

| Layer | Actual files/components | Responsibility and data movement |
|---|---|---|
| User/browser | Browser forms and downloads | Sends HTTP requests and renders responses; does not authorize itself |
| Presentation | `templates/*.html`, `static/css/style.css`, `static/js/charts.js` | Jinja-generated pages, custom styling, form helpers and Canvas charts |
| Request/security | `app.py`, `permissions.py` | Session lookup, CSRF, authentication, permission and ownership checks |
| Workflow/service | `workflows.py`, `revision_routes.py`, `services/` | Status rules, snapshots, payments, reservations, publication, reporting and documents |
| Persistence | `models.py`, Flask-SQLAlchemy, SQLite | Mapped records, transactions, relationships, indexes and constraints |
| Private file storage | Configured data-directory subfolders | Evidence, IDs and signature images; only protected routes expose selected files |
| Output/monitoring | DOCX generators, notifications, activity logs | Records or returns the authorized result; SMS/email currently simulated |
| Operations | `config.py`, `migration.py`, `seed.py`, `backup_sqlite.py` | Configuration, upgrade, initial catalogs/Admin, consistent database backup |

## 3.2 Technologies actually used

**Table 5. 3.2 Technologies actually used**

| Technology | Category | Purpose |
|---|---|---|
| Python | Programming language | Flask handlers, validation, models, services and maintenance utilities |
| JavaScript | Programming language | Form interaction, busy state, navigation, previews and Canvas charts |
| HTML / Jinja | Markup / templating | Server-rendered page structure and escaped content |
| CSS | Styling | Custom Navy and Civic Gold interface and responsive layout |
| SQL | Query language | Generated SQLAlchemy queries, SQLite pragmas and migration statements |
| SQLite / Python sqlite3 | Embedded database / standard library driver | Persistent local relational data and backup API |
| Flask 3.1.3 | Web framework | Routing, request/response handling and signed sessions |
| Flask-SQLAlchemy 3.1.1 / SQLAlchemy | ORM integration / library | Models, relationships, query construction and transactions |
| Werkzeug 3.1.8 | Web/security utility | Password hashing/checking and Flask infrastructure |
| python-dotenv 1.1.1 | Configuration library | Optional `.env` loading |
| python-docx 1.2.0 | Document library | Editable application DOCX output and this documentation's Word file |
| matplotlib 3.11.2 | Plotting library | Server-side chart PNGs for dashboards/reports and document figures |
| Gunicorn 26.2.0 | Deployment server | Configured Linux/Render WSGI deployment |
| unittest / Playwright with Edge | QA tools | Automated acceptance tests and browser audit; Playwright is a development tool |
| ReportLab / pypdf | Documentation-only tools | Build and structurally validate this PDF; not application runtime features |

There is no Flask-Login implementation: authentication is custom session-based code. Chart.js and Bootstrap are not used; charts use custom Canvas JavaScript and matplotlib, and layout uses project CSS. MariaDB/MySQL references in old material are legacy; `Config` rejects a non-SQLite `DATABASE_URL`.

## 3.3 Real project structure

```
My-projects--main/
  setup.ps1, run.ps1, render.yaml, RENDER_DEPLOYMENT.md
  SYSTEM_AUDIT_REPORT.md
  SYSTEM_RESEARCH_DOCUMENTATION.md       (historical)
  documentation/
    CAPSTONE_DOCUMENTATION.md            (current narrative source)
    collect_evidence.py, build_documentation.py
    source_evidence.json, assets/
  barangay_system/
    app.py, revision_routes.py
    models.py, permissions.py, workflows.py
    config.py, migration.py, seed.py, backup_sqlite.py
    requirements.txt, README.md
    services/
      analytics.py, documentgen.py, event_availability.py
      notify.py, revision.py, uploads.py
    templates/                           (Jinja pages and partials)
    static/css/style.css
    static/js/charts.js
    tests/                               (acceptance and audit utilities)
    instance/                            (private runtime data; ignored)
    audit-artifacts/                     (generated test evidence; ignored)
```

The `instance` directory contains the configured database and private data by default. It must not be distributed as sample project content. The documentation evidence manifest contains schema/source metadata only, not resident records or file contents from private storage.

## 3.4 Frontend behavior

`base.html` provides the authenticated shell, sidebar, header, notifications indicator and shared behavior. `auth_base.html` supports authentication pages. Page templates reuse cards, form rows, tables and status labels. Navy (`#082F5B`) emphasizes structure and Civic Gold (`#D4A62A`) marks highlights. CSS media rules and a mobile navigation toggle adapt the layout; tables may use scrolling containers.

Forms include CSRF tokens and browser-required controls, with backend validation remaining authoritative. Submission controls indicate a busy state and block repeated submission; restoration/timeouts recover controls. Feedback appears through flash/toast patterns. The code does not implement a uniform AJAX-based, section-local verification response for every ID/address/account action: those actions use redirects and flashes. Requirement notes remain associated with their requirement record, but uninterrupted inline review must not be promised.

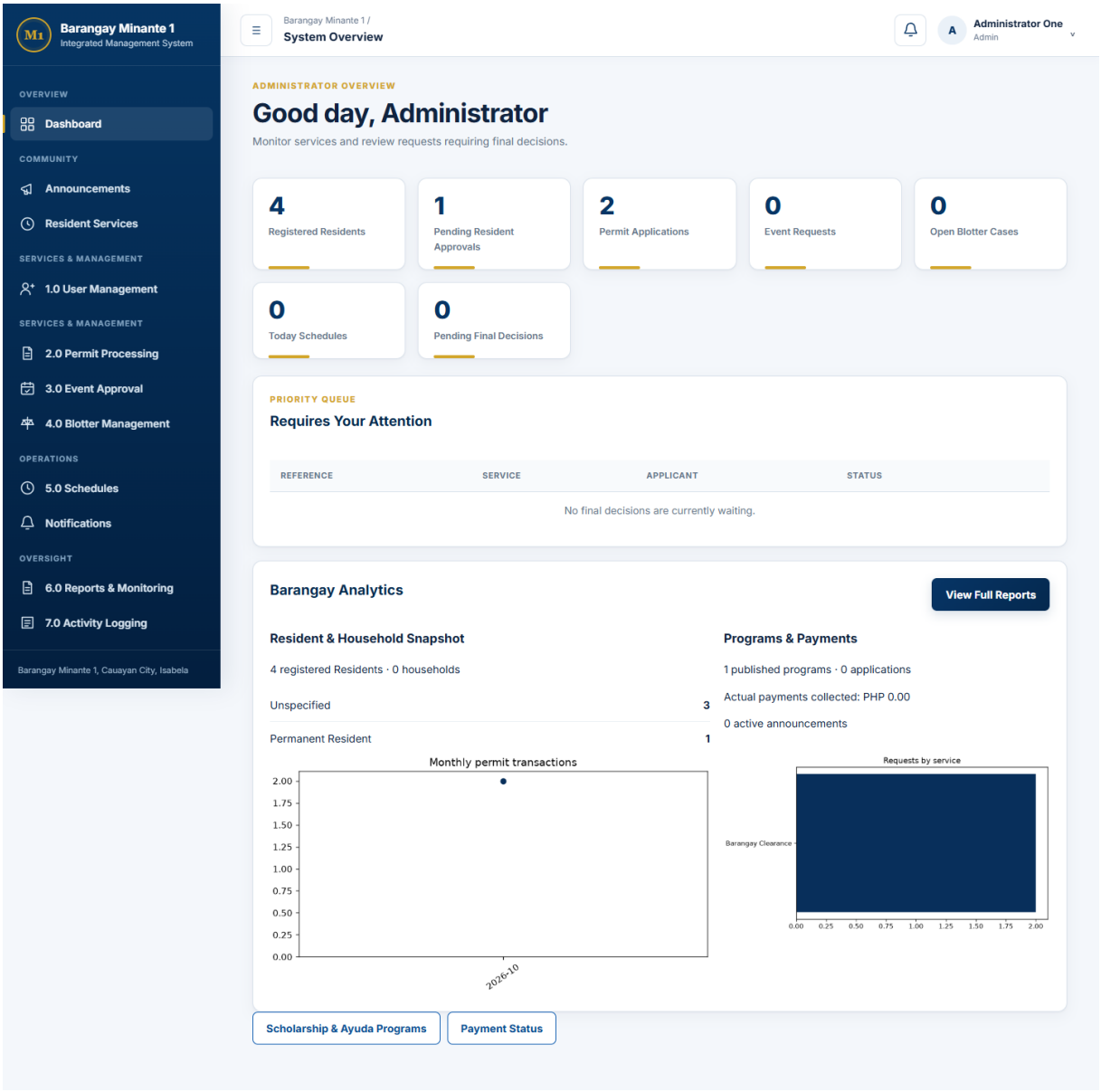



**Figure 4. Administrator dashboard from the synthetic browser audit**

This is an actual rendered interface using test fixtures, not production resident data. Displayed counts illustrate the audit fixture only; they are not population or performance statistics for Barangay Minante 1.

# Chapter 4 — Complete Feature and Functionality Documentation

## 4.1 Registration, account management and verification

**What / why / who.** Registration records an applicant's identity, residence and contact details so the Administrator can review access before service use. Public registration creates a Resident role; Administrator-controlled user management handles operational accounts.

**Simple explanation.** A person signs up, provides an accepted ID and address details, and waits for review. Submitting a registration does not immediately authorize login.

**Technical explanation / how.** `register()` validates personal/account fields, email/phone format, birth/residency values, password confirmation, required ID upload and privacy consent. `User` and `ResidentProfile` are saved together. New profiles have Pending approval and Pending Verification for ID/address. ID types must be active and accepted for registration. Optional “Others” selections require explanation. The registration sections include name, birth date, sex, civil status, occupation, house/street, Purok/zone, resident type, residency status, residency years, household information, emergency contact, username, email and password.

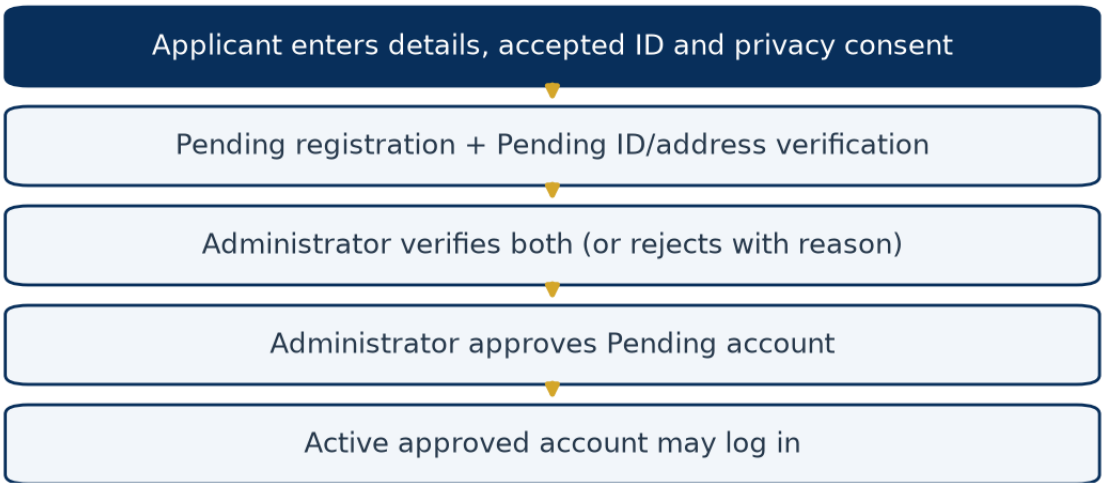



**Figure 5. Registration and account-access workflow**

`user_verification()` handles ID/address decisions and permits a decision only while its status is Pending Verification. A rejection requires a reason. `user_approval()` approves only a Pending registration after both verifications are Verified; repeat finalized decisions return a warning. `resident_identification_document()` allows the owner or Administrator, and uses a private directory. Image IDs can be previewed; PDFs are served as documents. Verification metadata includes actor, time and correction reason; `activity_logs` preserves important actions.

**Security / output / limitations.** A profile stores an ID file path and number; it does not validate the document against a government service. Changes to ID type/number/file through an authorized profile session reset ID review to Pending Verification. Pending/rejected applicants cannot use ordinary authenticated profile pages, so a complete pre-approval self-service correction loop is not present. The office must handle such cases; the documentation does not invent a correction route. ID/address/account decisions condition their database update on the expected Pending state. A stale competing action cannot change a finalized state through that update; review metadata and logs are written only after it succeeds.

## 4.2 Resident Type versus Residency Status

Resident Type describes the person's housing relationship; the Administrator-managed catalog is distinct from current Residency Status. The seeded types include Homeowner, Renter/Tenant, Household Member, Boarder/Bedspacer, Caretaker, Temporary Resident and Others. Residency Status records Permanent Resident, Temporary Resident, or Former Resident / Moved Out. A renter can be a permanent resident; therefore the two concepts should not be merged. `resident_profiles.resident_type_id` references the catalog while `residency_status` stores the current selected state.

Address review uses street/address, Purok/zone, residence category/status, years and household details displayed in account review. It records an administrative verification, not geolocation or proof generated by the software. Changes to street/Purok in `profile()` and approved address/Purok update requests reset `address_verification_status` to Pending Verification and clear previous reviewer/time/reason. This requests a fresh human decision; it does not automatically verify the new address. Changes to household text or residency category alone are not included in the same reset comparison. A complete pre-approval rejected-address correction portal remains absent.

## 4.3 Login, password recovery and profile

**Simple explanation.** Authentication identifies the account; authorization decides which actions that account may perform.

**Technical explanation.** `login()` looks up credentials, checks the Werkzeug password hash, checks activity and approval, records the session, then redirects to the role dashboard. `load_user_and_csrf()` loads the session user into `g.user`; `login_required()` rechecks role/activity/approval on protected requests. The browser receives a signed Flask session cookie, not a database password. Password validation requires at least ten characters with a letter and a digit. Profile password change additionally checks the current password.

`PasswordResetToken.issue()` creates a random token and stores its SHA-256 digest, expiry and consumption state. `valid_for()` checks the digest, unused state and expiry. Reset completion changes the password and marks outstanding tokens used. The default lifetime is 30 minutes. Recovery email is simulated; in debug mode a development reset link can be shown. This is not a working production email recovery delivery service. Sequential reuse is tested; do not describe all concurrent reset requests as atomically single-use.

`profile()` saves validated changes directly for an authorized logged-in user. This coexists with the Staff-assisted update-request approval feature; not all profile edits are routed through an approval queue. ID type/number/file changes restart ID verification; street/Purok changes restart address verification. These resets do not themselves change an already Approved account back to Pending approval.

## 4.4 Permit processing and dynamic requirements

**What / why / who.** Permits and certificates are configurable services requested by Residents or personally by Staff, reviewed by non-owner personnel, and finally decided by Administrators. Dynamic requirements allow service-specific evidence without hardcoding a separate form per certificate.

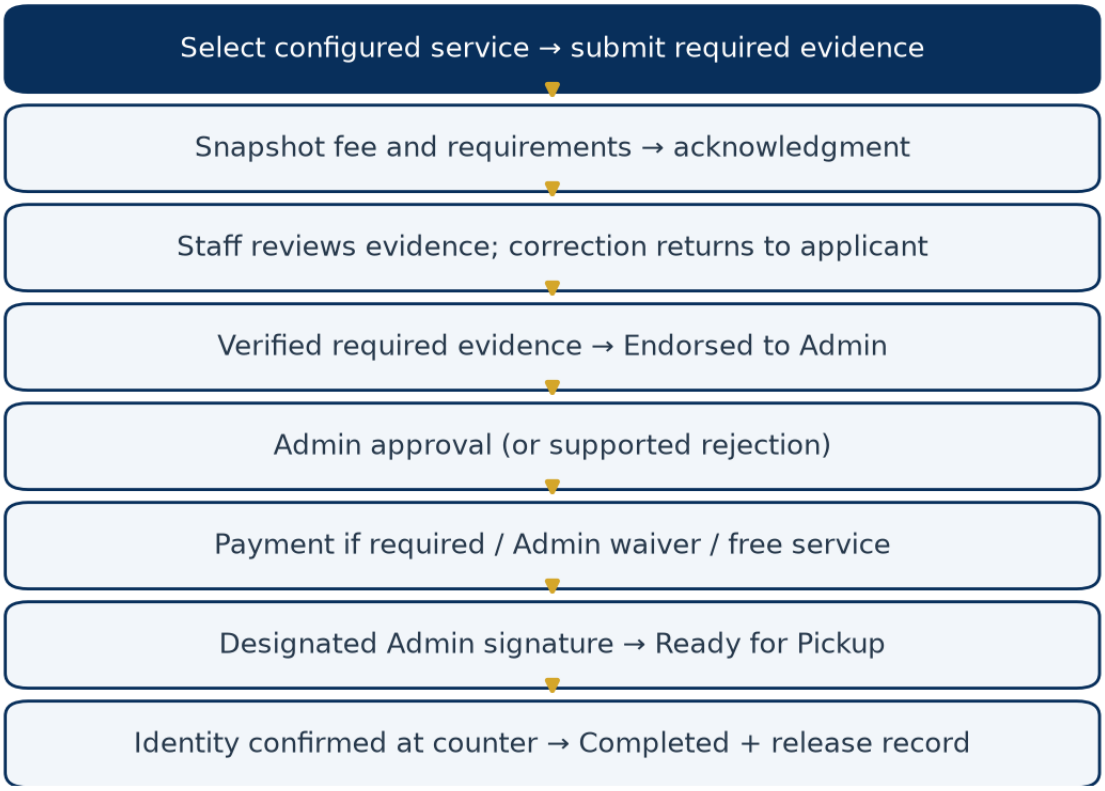



**Figure 6. Permit review, payment, signing and release workflow**

**Technical explanation / how.** `permits()` loads active permit types. A fee must be explicitly configured, including zero for a free service. Submission creates `permit_applications` and calls `create_requirement_snapshot()`. Each current definition becomes an `application_requirements` row with copied name, type, instructions and required flag, plus submitted text/file and state. `requirement_history` records evidence/review events. Required evidence is validated before commit; staged files are cleaned if the transaction rolls back.

Requirement types are Document (private PDF/PNG/JPEG), Text (typed value), or System Check. The implemented System Check tests the Resident profile's Approved account state; it is not a configurable external eligibility engine. Optional omissions are Not Submitted. Submitted items enter Submitted; review may move to Under Review, Verified or Needs Resubmission according to route validation. A correction requires a note, such as “Image is blurry; please upload a clearer copy.” The owner can replace evidence only when the workflow allows resubmission. A finalized Verified item is not simply reverified on repeat requests.

`requirement_progress()` counts submission/verification states for the progress display. Endorsement requires all required items to be Verified; optional omissions do not block it. `update_status()` validates the role-aware transition map. The main permit states are Pending, Under Review, Endorsed to Admin, Approved, Ready for Pickup and Completed, with supported Rejected/Cancelled branches. Generic state updates cannot bypass payment/signature/release routes.

**Database / security / output.** Definitions remain in `service_requirements`; the application copies preserve historical requirements even when future definitions change. `fee_at_submission` preserves the price at submission. `permit_processor_allowed()` prevents Staff processing their own permit. Owners can view their request and resubmit, but the current private requirement-download handler is for processing personnel, not a general owner evidence-download endpoint. A submission acknowledgment confirms intake; the official document becomes available only after approval, applicable payment and signature.

## 4.5 Pricing, electronic signature and approved document

The Administrator configures a non-negative service fee and readiness flag. A zero fee means Not Required for payment; a missing/unconfigured fee blocks new submission. Fee changes affect future submissions, not the stored `fee_at_submission` on existing records. A configured fee is not an actual collection.

`PermitSignatory` links a permit type to an authorized Administrator and private PNG/JPEG signature asset, with signatory name/title. `sign_approved_permit()` checks Approved status, verified required evidence, payment eligibility, correct Administrator and an existing signature image. It records signer/time, copied signatory details and document version, then sets Ready for Pickup. `generate_approved_permit_document()` builds native Word text/tables and includes the image when available. This is an electronic signature image with recorded authorization, not a certificate-based cryptographic digital signature or independently verifiable tamper seal. Editing the downloaded DOCX is technically possible.

## 4.6 Event approval and reservation conflict logic

**What / why / who.** Residents and Staff submit event requests for configured categories and venues. Personnel review and endorse; Administrators decide. Reservation checks coordinate competing requests for the same place and time.

**Actual inputs.** Event name, category, venue, description, proposed date, start time and end time. The category fee is snapshotted. Expected attendees and supporting attachment remain legacy database columns but are not current event form inputs.

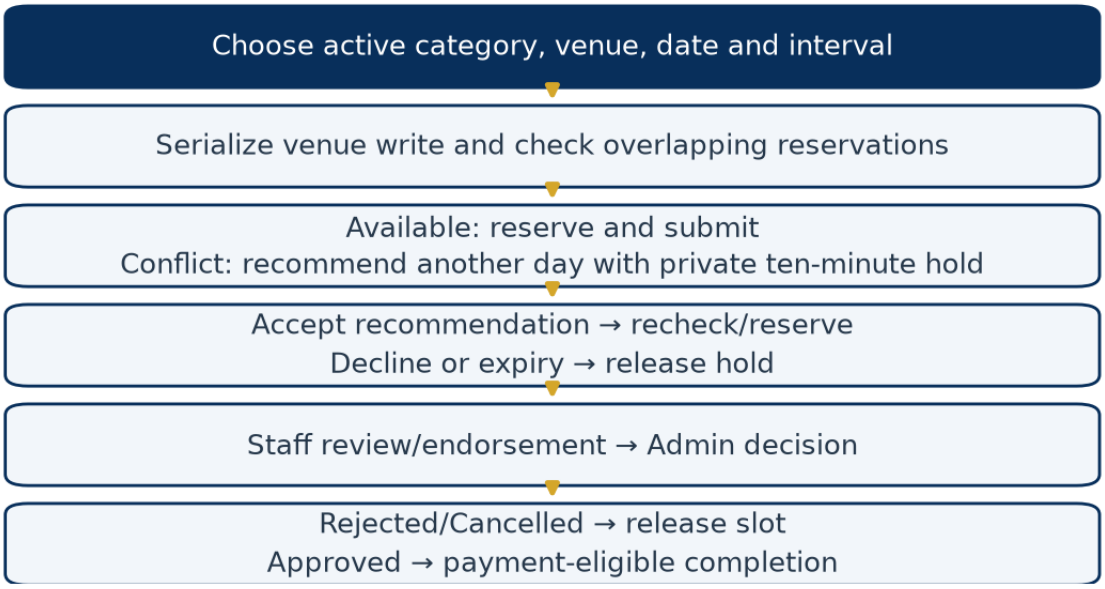



**Figure 7. Event request and private recommendation workflow**

**Technical explanation.** `_parse_event_form()` validates the active category/venue, configured fee and interval. `event_availability.py` serializes venue reservation writes using a venue update before conflict checks. `find_conflict()` compares intervals using start-before-other-end and end-after-other-start. Thus 09:00–10:00 and 09:30–10:30 overlap, while 09:00–10:00 and 10:00–11:00 are adjacent. Priority belongs to the transaction that successfully reserves the slot, not necessarily the first browser page opened.

`find_next_available()` searches alternative days with the requested time interval, up to 365 days; it does not search every free minute. `create_hold()` reserves a recommendation temporarily for ten minutes with an owner-bound hashed token and serialized draft. `accept_hold()` creates/links the real request after another conflict check. Decline, expiry, rejection or supported cancellation releases the reservation. Another applicant sees availability results, not the other person's event details. The current transition map allows cancellation only in its configured early state; no general Resident cancellation route is implied.

**Output / limits.** A successful submission produces a reference and acknowledgment; changes trigger notifications. Paid events need payment eligibility before completion. Event input is parsed as a naive datetime and compared with naive UTC helpers, unlike explicit Manila conversion for program/announcement inputs. Timezone treatment is therefore inconsistent and needs reconciliation before relying on all local-time boundaries.

## 4.7 Blotter management

**What / why / who.** A blotter records a complaint involving an identified respondent and incident. It differs from a general service concern such as a streetlight or drainage issue. Owners can file/view their own cases; Staff/Admin have operational access.

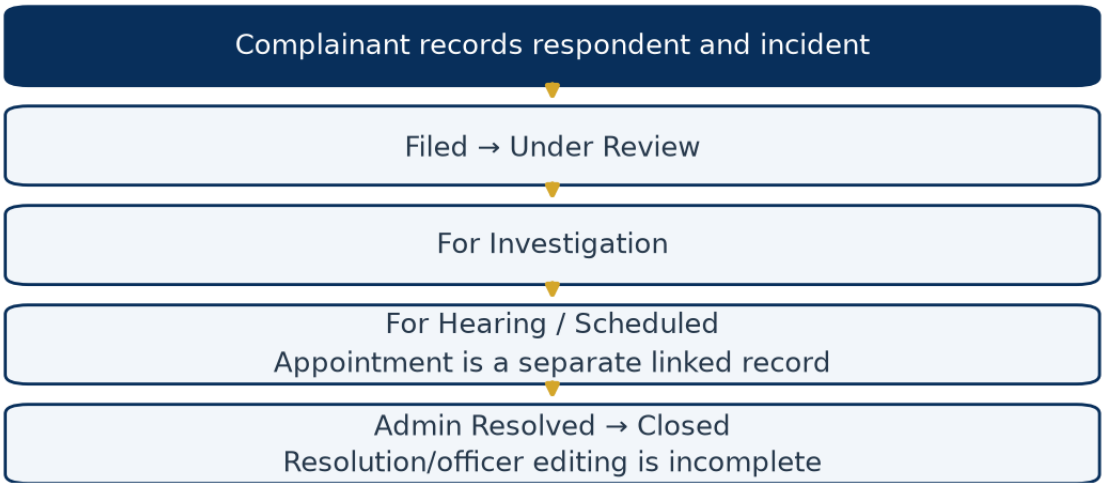



**Figure 8. Implemented blotter state progression**

`blotters()` records complainant account, respondent name/address, incident type/date/location, narrative and optional evidence. `blotter_detail()` applies the role transition map: Filed, Under Review, For Investigation, For Hearing / Scheduled, Resolved, then Closed where authorized. The `blotter_cases` table also contains `assigned_officer`, `hearing_date` and `resolution`, but the current detail POST delegates to generic status changes and does not provide a full editing workflow for these fields. A separate Schedule can reference the case, but setting the case to For Hearing / Scheduled does not itself create that appointment.

Private narratives are excluded from aggregate reports. The actual protection is role/ownership control, not a special investigator-assignment permission model. Resolution is a recorded processing state, not an automated legal determination.

## 4.8 Walk-in assistance, search and acknowledgments

**What / why / who.** Staff/Admin can assist residents who lack a device, connection or confidence using the system. The encoder and requester are different roles in the record.

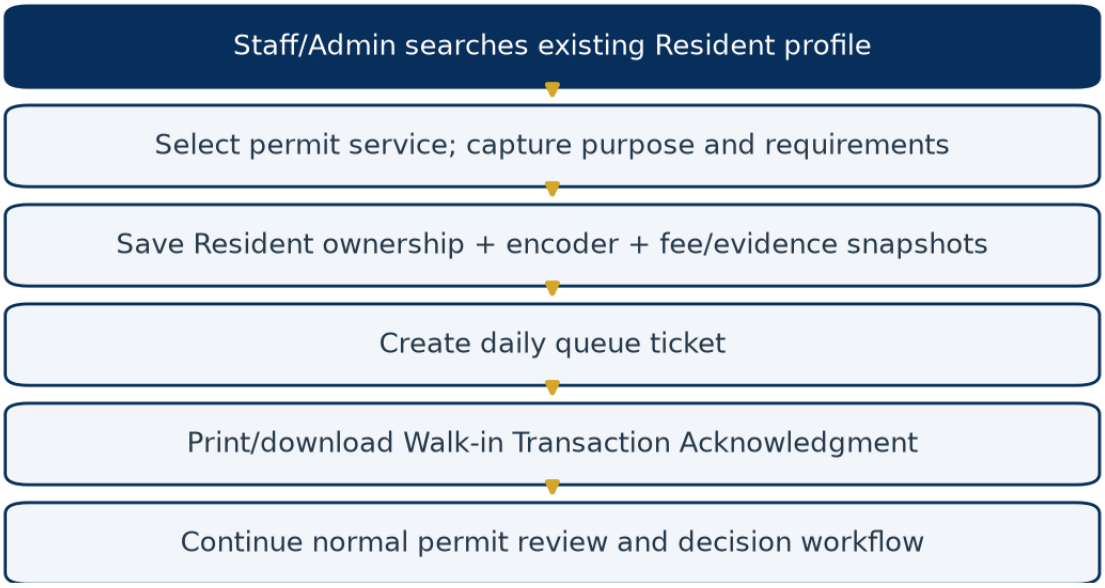



**Figure 9. Walk-in intake and queue workflow**

`search_residents()` searches name, username, numeric User ID, contact number, profile household number and Purok/zone; it returns at most 50 results. The quick selection card shows name, username and location/household summary, not a separate comprehensive profile application. Operational users select the resident, active permit service, purpose, optional counter and required evidence. `front_desk()` stores source "Walk-in / Staff Assisted," the Resident as `applicant_id`, and Staff/Admin as encoder. It snapshots requirements and creates a daily Document Request ticket in the same transaction.

The current front-desk resident eligibility helper requires a resident/staff account with a profile, but does not independently enforce active/Approved account status. A required System Check may reject an unapproved profile. Consequently the search's "approved" empty-state wording is stronger than the actual helper filter. This is a documented implementation gap, not an assurance of approved-only assisted submissions.

`walkin_acknowledgment()` provides the print view. The legacy-named `claim-stub.docx` route now returns a Walk-in Transaction Acknowledgment containing request/claim reference, resident, service, submission time, queue, requirements, fee/payment state, encoder, next steps and disclaimer. It does not approve the service or prove cash collection.

**Table 6. 4.8 Walk-in assistance, search and acknowledgments**

| Document | Trigger | Meaning |
|---|---|---|
| Submission Acknowledgment | Successful online request/program submission | Intake was recorded |
| Walk-in Transaction Acknowledgment | Assisted request; queue included for permit walk-ins | Office recorded an assisted transaction |
| Payment Receipt | Actual positive Paid row | Authorized personnel recorded payment |
| Approved Permit/Certificate | Approved, payment-eligible and signed request | Official generated service document in the implemented workflow |

Assisted program applications use an assisted acknowledgment but do not create a permit queue ticket. Full staff-assisted Event/Blotter encoder forms are not part of the dedicated walk-in permit form.

## 4.9 Queue, priority and release counter

Queue numbers use a category prefix and six-digit daily sequence; Document Request uses A, Complaint / Concern uses B, and the helper defaults to C for other categories. The current intake route creates Document Request tickets. A database unique constraint on `(queue_date, queue_number)` prevents duplicate daily identifiers. Queue creation is exercised by a two-request concurrency test; no claim is made for unlimited workload.

`update_queue()` implements Waiting → Now Serving → Serving → Completed. An update conditioned on the previous status rejects competing stale transitions. The display is ordered by creation time. Although `priority_category` exists, there is no active assignment/ranking engine for Senior Citizen, PWD, pregnancy or other categories. Personnel must follow Barangay policy manually; the documentation does not imply priority automation.

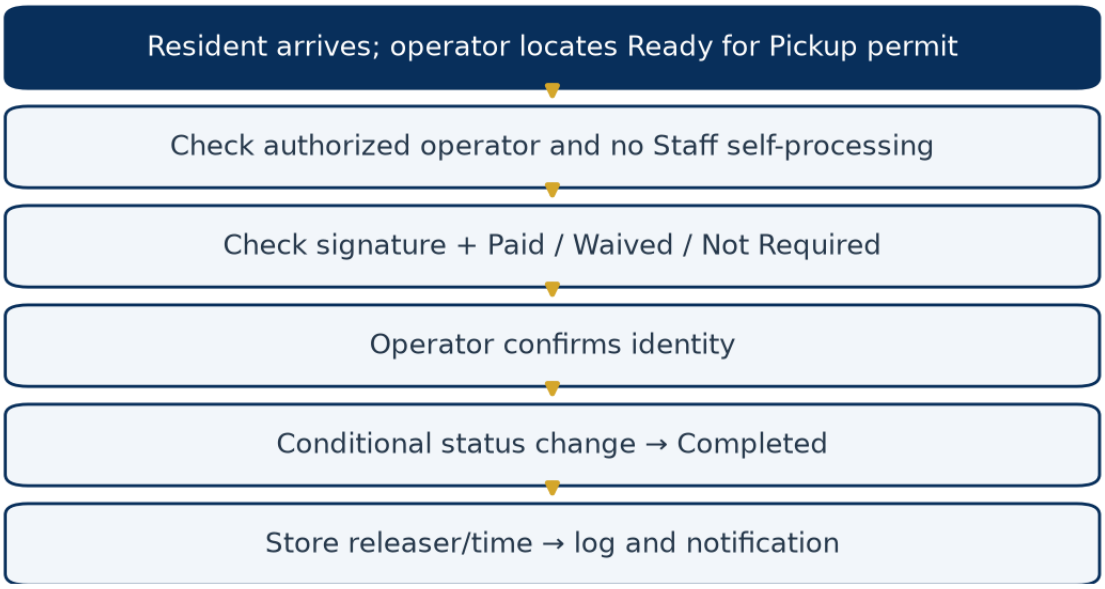



**Figure 10. Document release checks**

At release, Staff/Admin locate a Ready for Pickup permit, confirm identity, and submit the release action. `release_walkin_permit()` checks payment eligibility and signing, prevents Staff self-processing, and conditionally changes Ready for Pickup to Completed. It stores `released_by`/`released_at`, logs the action and notifies the owner. This route also supports ready online permits displayed at the counter. The identity confirmation is an operator checkbox, not biometric or live-ID validation. A second release attempt cannot repeat the eligible status update.

## 4.10 Appointments and assignment

`schedules()` creates a Requested appointment tied to an existing permit/event/blotter and its actual owner. Residents can choose only their own transactions; operational personnel can select operational records. Staff can Confirm, mark Rescheduled and Complete permitted states; Admin can additionally Cancel. Notifications record status changes. The schedule model has no separate office or officer-assignment fields; `confirmed_by` records who changed status, not an assigned officer.

**Partial implementation:** `schedule_status()` updates status but does not accept a replacement `scheduled_datetime`. Selecting Rescheduled therefore does not change the time. Do not instruct users to perform a nonexistent date-edit action.

Permit routing is separate: `assign_permit()` validates an active Staff/Admin assignee other than the requester, marks earlier assignments Reassigned, and creates an Active `request_assignments` row with actor/note. The permit's optional `office_name` is intake text, not a normalized office-routing system. Assignment history is stored;

it does not grant permission or restrict all other operational users from seeing the request.

## 4.11 Payment management

**Simple explanation.** Pricing says what should be paid; a Payment row records what authorized personnel say was actually received or waived.

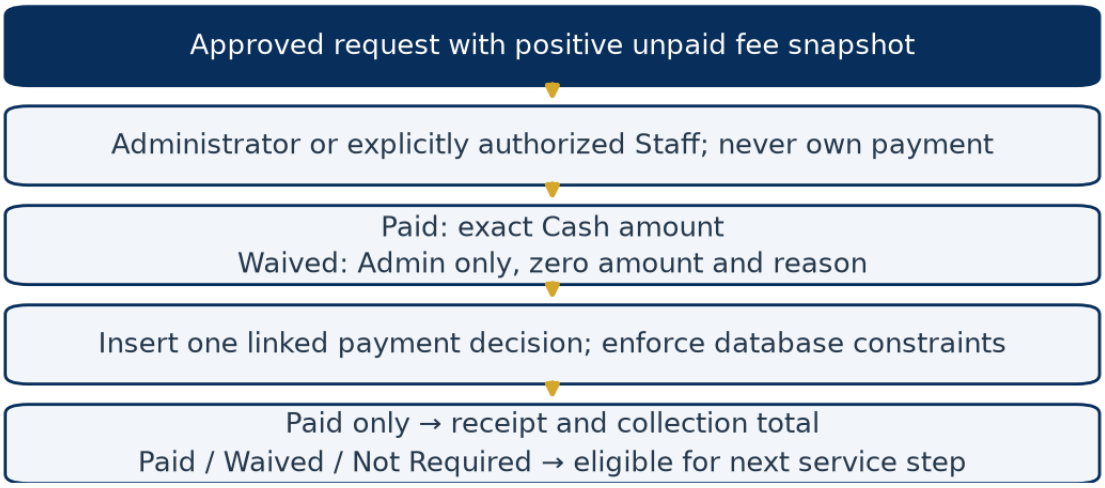



**Figure 11. Payment or waiver decision workflow**

`payments()` accepts manual decisions for approved unpaid permits/events. Administrators can grant/revoke Staff financial authority in `payment_authorities`. Ordinary Staff see their own payment records; authorized personnel can handle operational payments. No actor may record their own payment. Paid requires the exact positive fee snapshot and Cash method. Waived requires Administrator role, a reason and zero paid amount. There is no partial payment, refund, online gateway, bank reconciliation or extra-charge workflow.

`payments` links exactly one permit or event; unique request keys prevent another payment decision for the same request. Database checks allow only Paid/Waived with the correct amount relationship. `payment_status()` returns Not Required for a zero fee, otherwise Paid/Waived from the linked record or Unpaid. Only positive Paid records produce receipts or collection totals. A waiver is not a receipt. No application route edits historical payment decisions. Personnel with direct database access are outside that UI restriction; the ledger is not cryptographically immutable.

## 4.12 General concerns, referrals and resident updates

General concern intake supports free-text categories such as streetlight, waste, noise, drainage, road or stray-animal issues. These are examples, not a hardcoded official category catalog. `create_general_concern()` stores resident, description, location and Standard/Restricted marker. `route_general_concern()` assigns an active Staff/Admin and sets Routed. There is no full resolution/closure form. “Restricted” is a stored label; the front-desk role gate does not implement additional per-category clearance within Staff/Admin access.

`create_assistance_referral()` records a resident, category, description and sensitivity marker, with Open default status. Although an assignment column exists, there is no implemented end-to-end referral dispatch, assigned-personnel update, external agency response or closure route. It is intake, not emergency response or assistance delivery.

`create_resident_update_request()` is Staff/Admin assisted: it records old/proposed contact, email, address, household number or Purok values. Administrator `review_resident_update()` accepts Approved/Rejected only for Pending requests using a conditional update and validates email/phone for approval as applicable. Approved values update the target profile/User field. There is no Resident-facing update-request submission route; residents can still directly edit allowed fields through Profile. Household-number edits do not

automatically modify household membership records. Approved address/Purok edits restart address verification.

## 4.13 Household management

`households` stores unique household number, address, head and status; `household_members` links resident accounts with membership state and join/leave times. Staff/Admin `manage_household()` creates/finds a household, adds/reactivates membership, marks other active memberships Transferred, clears a transferred head and optionally sets the new head. It serializes changes before reading memberships and updates the resident profile's household-number text.

Actual membership rows, rather than profile text alone, determine Per Household program eligibility. One active membership is required. Household statistics aggregate stored rows. No dedicated move-out action is exposed; selecting Former Resident / Moved Out in a profile is not an automated membership termination. User-facing household text/head flags and normalized membership data can diverge through other edit paths, so operational reconciliation remains necessary.

## 4.14 Scholarship and Ayuda programs

**What / why / who.** Both program types use the same configurable engine. Staff/Admin create drafts; Admin approves/publishes/closes and makes final application decisions. Approved Residents can apply, and Staff/Admin can encode on behalf of an eligible Resident/Staff account.

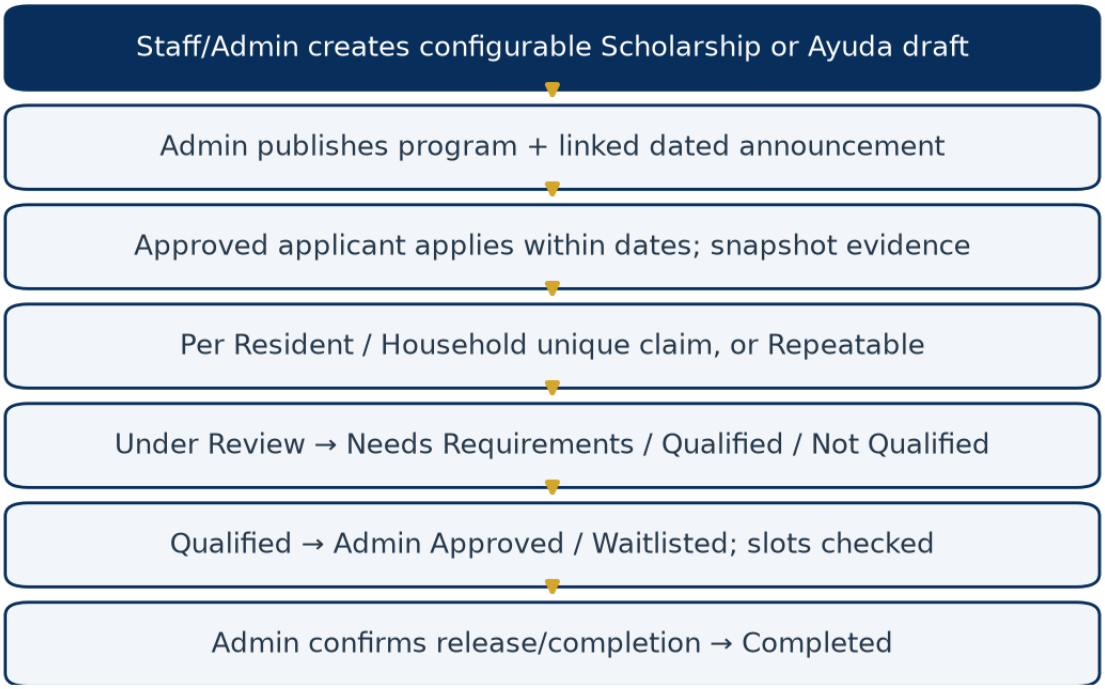



**Figure 12. Community program and application workflow**

Program configuration includes type, title, description, provider, assistance type, eligibility text, target categories, instructions, requirements, application start/deadline, optional distribution date/slots, application enablement and claim rule. The rule can be Per Resident, Per Household or Repeatable. Eligibility is reviewed manually; text criteria are not executable rules. Requirements use Document/Text/System Check validation, stored as JSON definitions and per-application snapshots.

Publication changes Draft to Published and creates a linked scheduled/active announcement with expiry at the application deadline. Resident visibility respects start time. Application submission requires Published status, enabled applications, an approved active applicant, and start ≤ now < deadline. Per Resident/Household creates a non-null claim key protected by `(program_id, claim_key)` uniqueness. Even a rejected application occupies the one-time claim key; no automatic reapplication exception is implemented. Repeatable uses a null key and

allows repeated applications.

Application states are Submitted → Under Review → Needs Requirements/Qualified/Not Qualified. A Needs Requirements application can be resubmitted. Qualified can become Approved or Waitlisted; Waitlisted can become Approved; Approved can become Completed. Negative/correction review requires a note. Qualification requires explicit evidence/eligibility confirmation. Admin-only approval checks slot capacity under a SQLite writer lock, counting Approved and Completed allocations. Completion requires explicit release confirmation and records releaser/time. Neither Staff nor Admin may process their own application.

`program_application_history` stores the prior evidence snapshot and resulting status/note for changes. Owners and authorized operators access private documents; program reports expose aggregates rather than applicant evidence. Ayuda completion records assistance release; Scholarship completion records fulfillment. Neither transfers money nor confirms receipt through an external agency. Closing a program archives linked announcements; application deadline enforcement does not automatically set the program's status to Closed.

## 4.15 Announcements and timed visibility

`announcements()` accepts title, body, category and required start/expiry timestamps. Staff submissions remain Draft; Admin creation can be approved immediately. Admin approves a still-valid draft to Scheduled/Published and can Archive via the compatibility `/delete` route, which preserves the row.




**Figure 13. Announcement lifecycle and preservation**

`local_datetime()` interprets naive announcement/program input as Asia/Manila and stores naive UTC. `active_announcements()` always applies start/expiry boundaries to the feed. `refresh_announcements()` persists Published/Expired changes and audit actions when relevant backend pages are accessed, with conditional updates for competing refreshes. No scheduler runs in the background, so persisted status/audit time may lag until the next access; visibility remains bounded by query time. Legacy announcements without an expiry retain prior behavior. Expired/Archived rows remain available in Administrator history. Linked Scholarship/Ayuda announcements lead to the program record.

## 4.16 Notifications, activity logs and feedback

`create_notification()` creates in_app, email or sms rows. In-app rows are marked sent and have read/unread state. Email/SMS with contact details are marked simulated; missing contact can produce failed. No provider API is called even if a provider-like environment value is set. Workflow helpers queue notifications with the same database transaction where applicable. `notification_read()` and `notifications_read_all()` update the current recipient's state. Operational send forms are available to Staff/Admin.

`audit()` calls `log_activity(commit=False)` to record actor, role, action, resource, reference, details, IP, user agent and UTC timestamp. Calling routes normally commit business changes and logs together. Login, account review, requirement review, payment, release, program changes, announcements and report actions have explicit log calls. Activity Logs is Admin-only, with no normal delete route. It is not a universal database change-capture service: actions without an explicit log call, such as current feedback creation, should not be described as independently audited. Logs are not cryptographically tamper-evident.

`submit_permit_feedback()` accepts a 1–5 rating and optional comment from the owner of a Completed permit. A `(resident_id, permit_application_id)` unique constraint blocks duplicates. The Administrator report shows counts and average rating. No survey-based satisfaction result is inferred from this facility.

## 4.17 Role dashboards and monitoring

**Administrator Dashboard.** `dashboard()` counts resident profiles, Pending resident approvals, permits, events, open blotters, today's appointments and endorsed requests awaiting final decisions. `dashboard_analytics()` adds household/program/application totals, actual Paid collections and active announcements. “Registered Residents” is a profile count, which can include Staff with resident profiles. The actionable tables filter endorsed requests before taking the latest six, so unrelated completed records do not hide older pending decisions.

**Staff Dashboard.** Counters cover other applicants' Pending/Under Review/endorsed permits and events, new blotters, today's appointments, personal requests, ready documents, requirements needing correction, unresolved concerns and today's assisted permits. Review tables exclude Staff-owned permit/event requests. Personal requests remain separately accessible. A counter is a count of stored records, not a ranking of employee performance.

**Resident Dashboard.** Queries are scoped to the current owner's permits, events, blotters and schedules. Cards show active applications, blotter count, upcoming appointments and unread in-app notifications. The latest implementation excludes past and Cancelled appointments from the upcoming list using Manila wall time and the allowed appointment states. Recent personal requests and notifications provide navigation to their detail pages.

**Code / data / controls.** The route reads the relevant models and supplies `dashboard.html`; it does not maintain a second dashboard database. Full analytics are passed only for the Administrator. The stabilization tests cover older actionable work remaining visible and Resident appointment filtering. The historical screenshot in Figure 4 uses synthetic records and may show fewer entries than a live installation.

# Chapter 5 — SQLite Database Documentation

## 5.1 Database purpose and connection

**Simple explanation.** SQLite is the application's digital filing cabinet. It stores related records in a persistent file so closing a browser or restarting the process does not erase them when the file is on persistent storage.

**Technical explanation.** `Config.SQLALCHEMY_DATABASE_URI` uses a SQLite connection string. By default the database is `barangay_system/instance/minante.db`; `DATA_DIR` or `SQLITE_DB_PATH` can change this location. Flask-SQLAlchemy attaches a session to Flask and maps classes in `models.py` to tables. SQLAlchemy sends SQL through Python's sqlite3 driver. SQLite was selected in the current configuration for a small, single-service deployment with no separate database server. This is an architectural fit, not evidence of a formal comparison study.

The connection hook enables foreign keys, a 30-second busy timeout, WAL journaling and NORMAL synchronous mode. WAL helps readers coexist with a writer; it does not enable unlimited parallel writers. `check_same_thread=False` permits connection use managed by the application stack; it does not make arbitrary shared-session access safe. ORM sessions must remain request/thread scoped.

## 5.2 Primary keys, foreign keys and relationships

A primary key uniquely identifies a row: `users.id` identifies an account. A foreign key links records: `permit_applications.applicant_id` points to `users.id`. Usernames and public references are business identifiers protected by unique constraints; they do not replace all internal numeric keys.

One User can have many permits, events and blotters, but at most one ResidentProfile because `resident_profiles.user_id` is unique. One permit has many snapshotted requirements; each requirement has history. A payment links one permit or one event, never both. A program has many applications and each application has history. A household has many membership rows. Repeated actor references (creator, reviewer, releaser) point back to Users and have distinct meanings.

`schedules.related_type/related_id` and notification related references are polymorphic logical links validated in routes; they are not direct foreign keys to three different transaction tables. `resident_profiles.household_number` is text, not a foreign key to `households`; operational membership is represented by `household_members`. Valid ID data lives in ResidentProfile plus private files, not a separate invented Identification table.

## 5.3 Simplified entity relationship diagrams

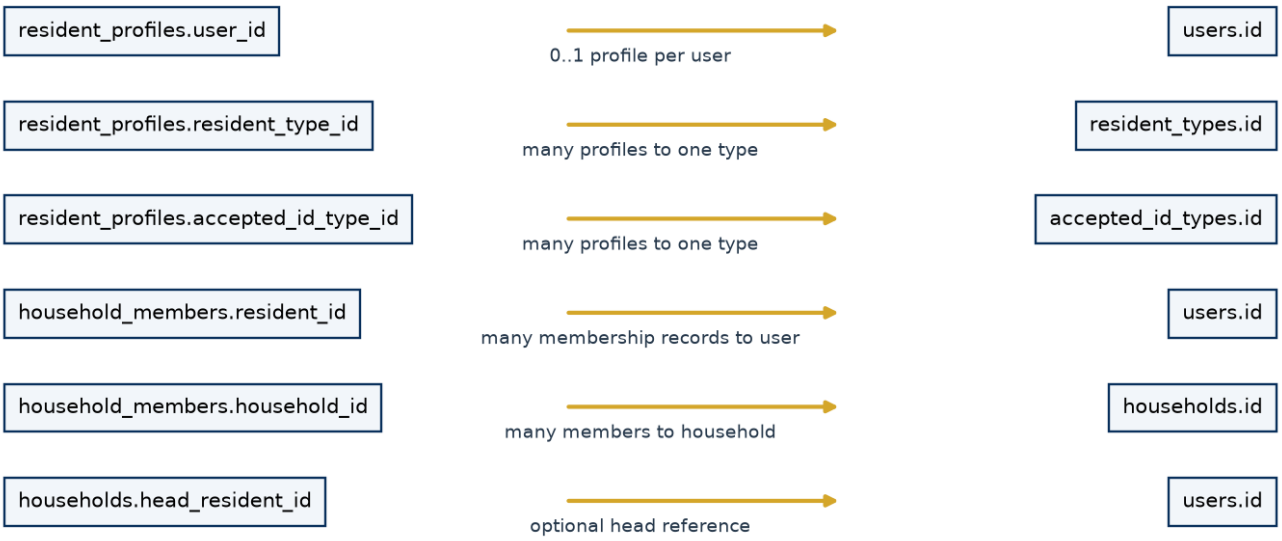



**Figure 14. Resident, identity and household relationships**

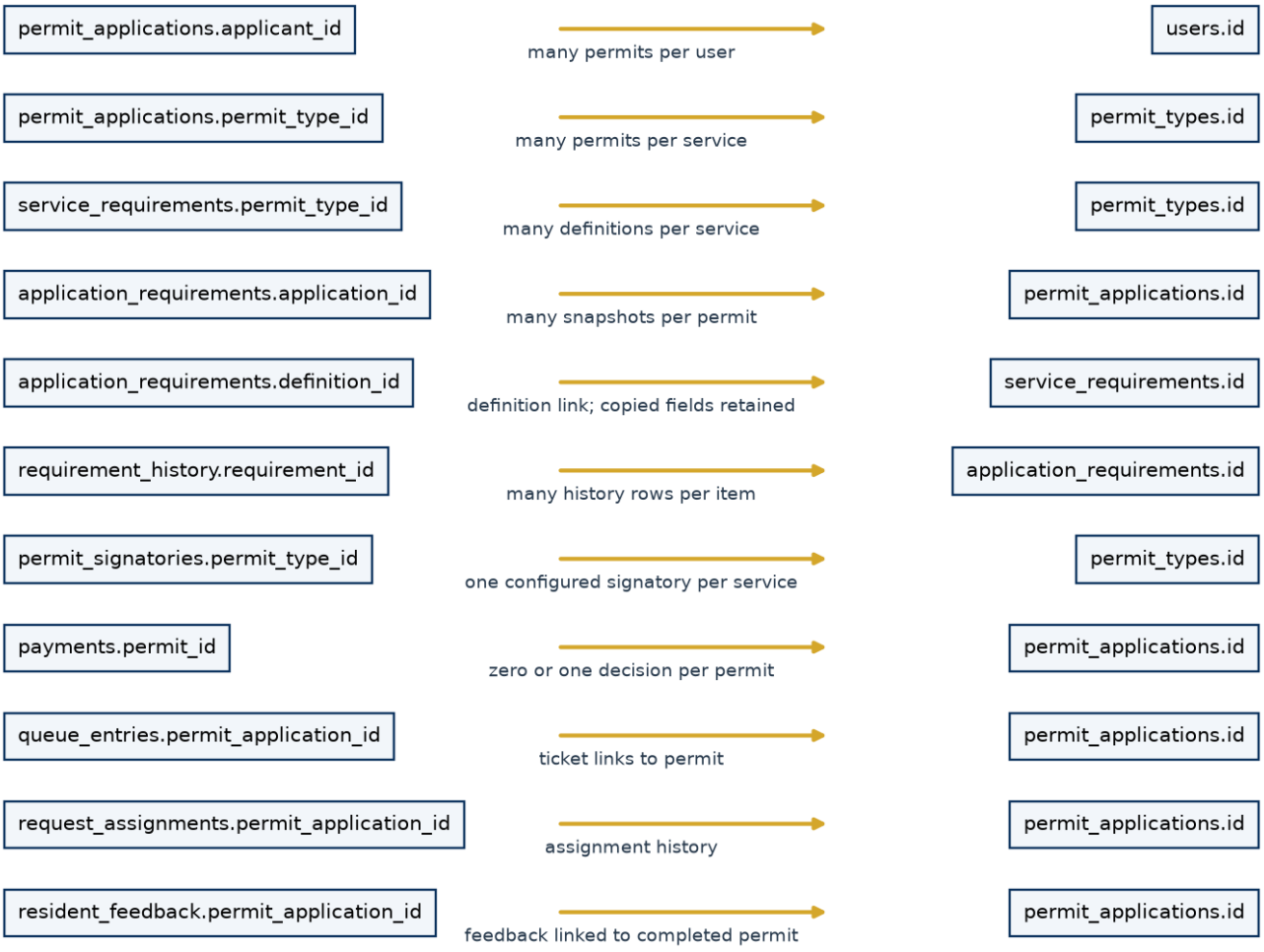



**Figure 15. Permit, requirements, queue, signature and payment relationships**

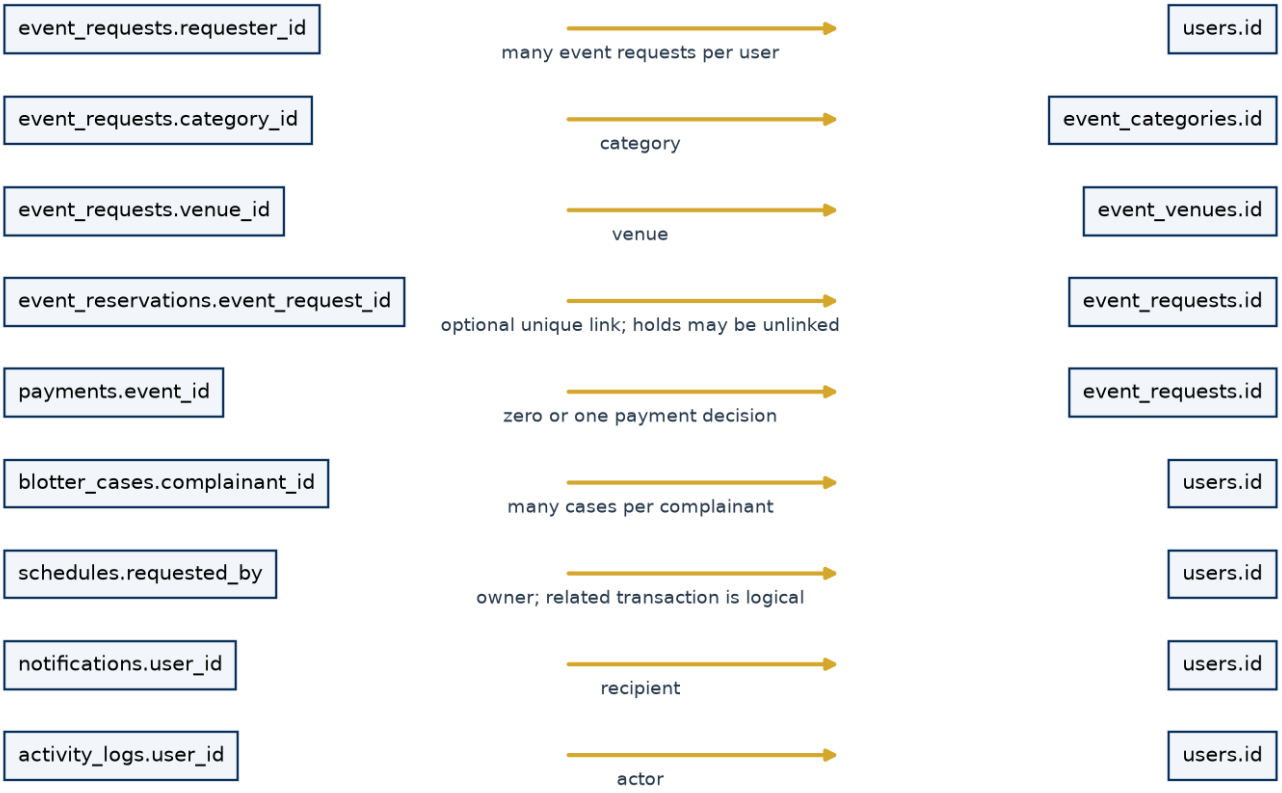



**Figure 16. Events, blotters and shared operational records**

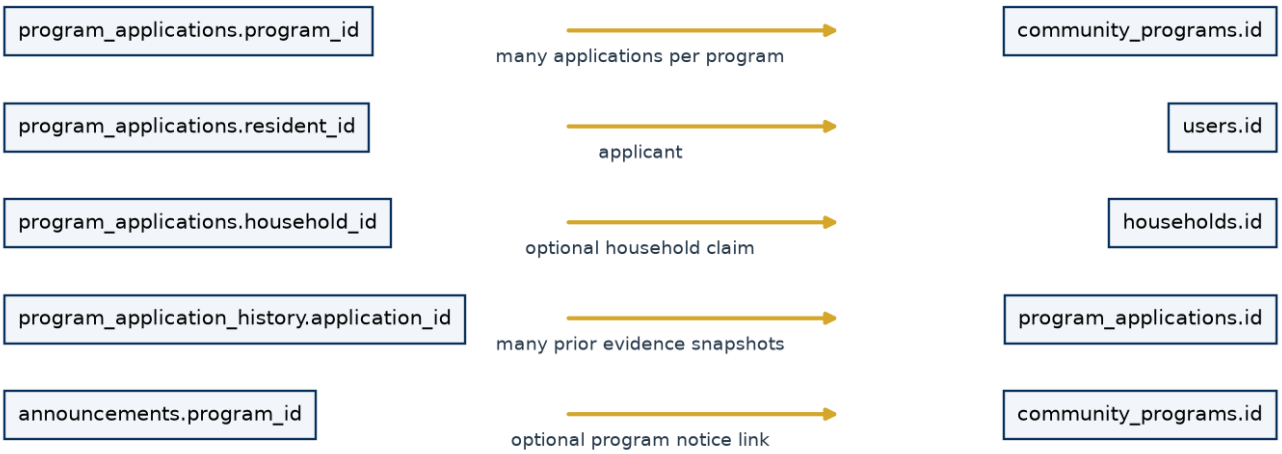



**Figure 17. Programs, applications, history and announcements**

Arrows in these diagrams point from a child foreign-key column toward its referenced parent. They show principal links; repeated actor links are summarized to avoid unreadable diagrams. Polymorphic logical links are discussed in the text and are not drawn as physical foreign-key arrows. The complete column and foreign-key dictionary appears in Appendix A.

## 5.4 Database table catalog

**Table 7. 5.4 Database table catalog**

| Table | Purpose / module | Data created/updated by |
|---|---|---|
| accepted_id_types | ID catalog with active and accepted-for-registration flags | Migration/Administrator configuration |
| activity_logs | Explicit actor/action/resource/reference audit records | Instrumented handlers; Admin reads; no ordinary delete route |
| announcements | Dated notices, categories, approval and optional linked program | Staff draft/Admin publication; lifecycle refresh/archive |
| application_requirements | Copied requirement definitions and current submitted evidence | Submission snapshot; eligible owner resubmission and personnel review |
| assistance_referrals | Assistance referral intake with sensitivity/status fields | Staff/Admin intake; later assignment/closure not exposed |
| blotter_cases | Complainant/respondent and incident record with case state | Resident/Staff/Admin intake; authorized status processing |
| community_programs | Scholarship/Ayuda definitions, dates, claim rules and requirements | Staff/Admin draft; Admin publish/requirements/close |
| event_categories | Event category and configured fee catalog | Seeder/Administrator configuration |
| event_requests | Event applicant, interval, category/venue, fee and decisions | Resident/Staff submission; non-owner Staff/Admin processing |
| event_reservations | Held/reserved venue intervals, private tokens and release state | Availability service during submit/accept/decline/expiry/decision |
| event_venues | Configured active venues | Administrator configuration; reservation lock performs a no-op update |
| general_concerns | Resident service concern intake and personnel routing | Staff/Admin intake and routing; no complete closure route |
| household_members | Resident membership, transfer and join/leave metadata | Staff/Admin household management |
| households | Household number, address, head and status | Staff/Admin household management |
| notifications | Recipient/channel/content, delivery simulation and read state | Workflow/authorized sender; recipient read actions |
| password_reset_tokens | Hashed expiring and consumed recovery tokens | Recovery issue/reset handlers |
| payment_authorities | Explicit Staff financial permission grants | Administrator grant/revoke |
| payments | Actual payment or waiver linked to exactly one service request | Authorized Admin/Staff payment handler; no edit route |
| permit_applications | Permit ownership, snapshot fee, lifecycle, signature and release data | Resident/Staff/Admin submission; authorized review/sign/release handlers |
| permit_signatories | Per-service authorized Admin and protected signature configuration | Administrator configuration |
| permit_types | Permit/certificate catalog, configured fee and processing information | Seeder/Administrator configuration |
| program_application_history | Previous evidence snapshots and recorded program transitions | Program review/resubmission handlers |
| program_applications | Applicant/household claim, snapshot evidence and lifecycle | Owner or assisted submission; authorized review/Admin completion |

| Table | Purpose / module | Data created/updated by |
|---|---|---|
| queue_entries | Daily assisted permit tickets and stage timestamps | Front desk intake; Staff/Admin queue actions |
| reports | Generated report type/format/filter metadata | Administrator DOCX export |
| request_assignments | Permit assignment history and current Active assignment | Staff/Admin assignment route |
| requirement_history | Evidence/review history for individual permit requirements | Snapshot/resubmission/review handlers; no ordinary history editor |
| resident_feedback | One rating/comment per resident and completed permit | Eligible owner submission; aggregate report reads |
| resident_profiles | Residence, ID/address review, consent and account-approval metadata | Registration/Admin/migration; owner and Administrator review |
| resident_types | Configurable housing relationship catalog | Migration/Administrator configuration |
| resident_update_requests | Old/proposed field values and update decision | Staff/Admin proposal; Administrator approval/rejection |
| schedules | Transaction-linked appointments and processing states | Owner or operator creation; Staff/Admin status updates |
| service_requirements | Future permit requirement definitions and order | Administrator configuration |
| system_settings | Key/value operational settings | Administrator configuration/migration |
| users | Accounts, password hashes, roles and contact data | Registration/Admin creation; owner profile and Admin account management |

## 5.5 Constraints and integrity

The inspected live schema contains 35 application tables, with no observed SQLite integrity errors or foreign-key violations at the read-only check. This is a point-in-time structural check and does not certify that every operational record is factually correct.

Important constraints include unique usernames/emails and public references; one ResidentProfile per User; unique daily queue numbers; one payment decision per permit/event; exactly one payment target; valid Paid/Waived amounts; one one-time claim key per program; and one feedback response per resident/permit. Program tables also check program kind, status, positive slots and date ordering. Some rules are enforced in routes rather than SQL, including allowed workflow transitions and many text/value choices. SQLite string-length declarations do not by themselves provide the same validation as every form rule.

## 5.6 CRUD with actual examples

**Table 8. 5.6 CRUD with actual examples**

| Operation | Simple meaning | Current example |
|---|---|---|
| Create | Store a new row | Add PermitApplication and its ApplicationRequirement snapshots |
| Read | Retrieve stored rows | Show only the current Resident's requests; aggregate Paid amounts |
| Update | Change eligible state/data | Verify evidence or mark a ready permit Completed at release |
| Delete | Remove a row | Revoke a Staff PaymentAuthority row |

Not every module offers all CRUD operations. Announcements use archival instead of deletion; financial decisions have no edit/delete interface. This preserves operational history without claiming the database file cannot be

modified by an administrator of the host.

## 5.7 Transactions, commit and rollback

**Simple explanation.** A transaction keeps related changes together. Commit saves the whole successful unit; rollback discards unsuccessful database changes.

**Technical explanation.** The SQLAlchemy session accumulates additions and updates. `flush()` sends pending SQL and supplies generated IDs while leaving the transaction open; it is not the final save. Permit submission flushes the parent, creates snapshots/history/notifications and commits together. If required evidence fails, rollback prevents a half-created request. `services/uploads.py` tracks staged files because database rollback alone cannot remove an already-written file.

Event reservation takes the SQLite writer lock before checking availability and saving the reservation. Payment recording uses unique target keys and catches IntegrityError to reject competing duplicates. Program slot approval acquires a writer lock before counting allocated slots. Queue/release/program, resident-update and ID/address/account decisions use conditional updates against their expected prior state. Several generic status handlers still use ordinary read-then-write checks; concurrency guarantees should not be generalized to every route.

## 5.8 Queries, indexes and persistence

The ORM builds bound-parameter queries for normal user values; migration identifiers come from trusted schema definitions. List pages use filters and pagination. Indexes cover ownership/date lists, approval/status searches, notification owners, references, reservation intervals and program applications. The measured synthetic Permit query used `ix_permit_owner_date`. SQL aggregates (`COUNT`, `SUM`, grouping and date functions) support reports without exporting private case narratives.

Persistent storage must include the SQLite file and private directories. Render's supplied demonstration configuration uses an ephemeral path and therefore cannot guarantee retention across redeploys. SQLite database files are not encrypted by this application; host permissions, persistent storage access and backup protection matter.

## 5.9 Initialization and additive migration

`upgrade_legacy_schema()` creates missing mapped tables, adds known missing columns, creates declared indexes and normalizes selected legacy data. It is designed to be additive and idempotent; it does not drop/rebuild historical tables. It also creates required reference catalogs and compatibility resident profiles. Legacy trusted accounts may be normalized to verified/approved states; this is migration compatibility, not fresh verification evidence.

Adding a column to an old SQLite table does not automatically recreate every model-level CHECK or foreign-key constraint. This document's dictionary separates the actual inspected SQLite foreign keys from ORM declarations. Fresh and upgraded installations should be inspected after migration; schema version history is not managed by Alembic here. Legacy payment flags are not converted into invented payments. Old released/paid records without a linked Payment may require manual reconciliation.

# Chapter 6 — Code and Logic Reference

## 6.1 How to follow a request through the source

Read the matching route in `app.py` or the nested handler registered by `revision_routes.py`; identify its login and permission checks; follow validation/services; find the model columns; then inspect the named template and tests. `permissions.py` defines general grants, while `workflows.py` defines core status transitions. Route-level checks are equally important: payment authority, ownership, designated signing and program processing are not fully described by the general grant table.

The inventory in Appendix B records all 72 source-declared routes with methods, handler and file location. Flask adds its static-file route, producing the 73-route runtime inventory seen in the audit. It is not a public REST API specification: most endpoints return HTML, redirects or document streams.

## 6.2 Important functions/classes

Each entry below states its purpose, input, processing, output, database interaction, authorization context, possible errors and feature. Line references in the generated source index help locate the exact inspected version.

### load_user_and_csrf

**Code:** `barangay_system/app.py:367`. **Input:** Session user ID, HTTP method and CSRF field/header.

**Purpose / processing / output:** Loads g.user; creates/checks token before route execution. **SQLite interaction:** Reads User.

**Permission / possible errors:** Public request hook; invalid mutation token yields 400.

### login_required

**Code:** `barangay_system/app.py:381`. **Input:** Protected view and current actor.

**Purpose / processing / output:** Redirects unauthorized account or runs view. **SQLite interaction:** User/Profile activity and approval lookup.

**Permission / possible errors:** Checks allowed role, activity and approval; login redirect on failure.

### permission_required

**Code:** `barangay_system/permissions.py:34`. **Input:** Resource/action and current user.

**Purpose / processing / output:** Wraps a route using can() grants. **SQLite interaction:** No direct write.

**Permission / possible errors:** Missing user 401; missing grant 403.

### user_verification

**Code:** `barangay_system/app.py:2608`. **Input:** Resident ID, id/address, verify/reject and reason.

**Purpose / processing / output:** Conditionally finalizes Pending Verification; logs actor/time. **SQLite interaction:** Updates ResidentProfile and ActivityLog.

**Permission / possible errors:** Admin only; missing ID file/invalid repeat produces warning; reason needed for rejection.

### user_approval

**Code:** `barangay_system/app.py:2677`. **Input:** Resident ID and approve/reject decision.

**Purpose / processing / output:** Conditionally finalizes Pending approval; queues notification. **SQLite interaction:** Profile, notifications and logs commit together.

**Permission / possible errors:** Admin only; approval requires both verification states still Verified.

### create_requirement_snapshot

**Code:** `barangay_system/app.py:252`. **Input:** Permit, current definitions, form/files and resident.

**Purpose / processing / output:** Copies definitions; validates typed evidence; creates history. **SQLite interaction:** ApplicationRequirement and RequirementHistory insert.

**Permission / possible errors:** Called from authorized intake; ValueError rolls back incomplete submission.

### requirement_submission

**Code:** `barangay_system/services/revision.py:73`. **Input:** Definition, form/files, upload helper and resident.

**Purpose / processing / output:** Returns value, private path and state for Document/Text/System Check. **SQLite interaction:** Reads profile; upload staged separately.

**Permission / possible errors:** Required absence/type error; System Check uses approved profile.

### update_status

**Code:** `barangay_system/app.py:2283`. **Input:** Transaction, kind, owner and target status.

**Purpose / processing / output:** Checks role transition and prerequisites; stores status/notifies. **SQLite interaction:** Request, event reservation if released, logs and notifications.

**Permission / possible errors:** Staff cannot process own; 400 invalid transition, 409 missing prerequisite.

### reserve_event

**Code:** `barangay_system/services/event_availability.py:103`. **Input:** Event record, venue, start/end.

**Purpose / processing / output:** Locks venue write, rechecks conflict, returns staged reservation. **SQLite interaction:** EventVenue update and EventReservation insert/flush.

**Permission / possible errors:** Service called after route authorization; ReservationConflict/ValueError.

### create_hold

**Code:** `barangay_system/services/event_availability.py:122`. **Input:** Owner, venue, recommended interval and draft.

**Purpose / processing / output:** Returns raw private token and ten-minute held reservation. **SQLite interaction:** Stores token digest, expiry and draft JSON.

**Permission / possible errors:** Owner-bound later access; conflicting interval rejected.

### payments

**Code:** `barangay_system/revision_routes.py:48`. **Input:** GET list or POST request kind/ID, amount, method, decision.

**Purpose / processing / output:** Scopes visible records or records exact payment/waiver. **SQLite interaction:** Payment plus logs/notifications; unique target enforced.

**Permission / possible errors:** Financial authority; no own payment; Admin waiver; 400/403/409.

### sign_approved_permit

**Code:** `barangay_system/app.py:2251`. **Input:** Approved permit ID and signed-in Admin.

**Purpose / processing / output:** Checks configuration/payment/evidence; sets Ready for Pickup. **SQLite interaction:** Copies signature metadata to PermitApplication; logs/notifications.

**Permission / possible errors:** Designated Admin; 400/403/409 on unavailable or invalid state.

### release_walkin_permit

**Code:** `barangay_system/app.py:1198`. **Input:** Ready permit ID and identity_confirmed=1.

**Purpose / processing / output:** Conditional Completed update records release actor/time. **SQLite interaction:** PermitApplication, logs, notifications.

**Permission / possible errors:** Staff/Admin except Staff own; checks payment/signature; stale release rejected.

### program_application

**Code:** `barangay_system/revision_routes.py:283`. **Input:** Application ID, target, note and evidence/release confirmation.

**Purpose / processing / output:** Validates state; replaces evidence or reviews/allocates/completes. **SQLite interaction:** Application/history; program lock for capacity; notifications/logs.

**Permission / possible errors:** Owner resubmit; operator review; Admin final steps; owner cannot process.

### active_announcements

**Code:** `barangay_system/services/revision.py:41`. **Input:** Current UTC clock.

**Purpose / processing / output:** Returns approved currently visible announcement query. **SQLite interaction:** Filters Announcement start/expiry/status.

**Permission / possible errors:** Caller scopes use; no background scheduler.

### refresh_announcements

**Code:** `barangay_system/services/revision.py:51`. **Input:** Audit callback and current UTC clock.

**Purpose / processing / output:** Conditionally persists Published/Expired boundary changes. **SQLite interaction:** Announcement update plus logs and commit.

**Permission / possible errors:** Invoked from backend pages; inactive periods may delay persisted state.

### build_report

**Code:** `barangay_system/services/analytics.py:50`. **Input:** Validated report filters.

**Purpose / processing / output:** Builds aggregate sections for report/chart/export. **SQLite interaction:** SQL count/sum/group/date queries across selected models.

**Permission / possible errors:** Admin enforcement at caller; incorrect filters rejected earlier.

### word_report

**Code:** `barangay_system/services/analytics.py:238`. **Input:** Aggregate report and generating user display name.

**Purpose / processing / output:** Returns BytesIO DOCX with text, tables and chart pictures. **SQLite interaction:** No direct query; caller builds report and metadata/logs.

**Permission / possible errors:** Admin export caller; invalid image/document errors propagate.

### PasswordResetToken

**Code:** `barangay_system/models.py:543`. **Input:** User/lifetime or raw token.

**Purpose / processing / output:** Issues hashed token; finds unused unexpired token. **SQLite interaction:** Token insert/update and User password reset by handler.

**Permission / possible errors:** No live delivery; invalid/expired link redirects at route.

### upgrade_legacy_schema

**Code:** `barangay_system/migration.py:354`. **Input:** Configured app/database context.

**Purpose / processing / output:** Creates missing schema/indexes and normalizes legacy records. **SQLite interaction:** Schema DDL and compatibility updates.

**Permission / possible errors:** Maintenance/startup use; back up first; no claim all constraints rebuilt.

### backup

**Code:** `barangay_system/backup_sqlite.py:9`. **Input:** Config.SQLITE_DB_PATH and DATA_DIR.

**Purpose / processing / output:** Online backup API, integrity check and atomic final rename. **SQLite interaction:** Reads source, creates separate backup DB.

**Permission / possible errors:** Operator file access; missing file/integrity failure raises RuntimeError.

## 6.3 Selected short code examples

The following snippets explain actual code; the full files remain the authoritative implementation.

```
# services/revision.py: payment_eligible
def payment_eligible(item):
    return payment_status(item) in {"Not Required", "Paid", "Waived"}
```

This central rule lets a free service or authorized waiver progress without inventing a Paid record. Signing, official downloads and release call this rule.

```
# permissions.py: validate a general grant
def can(role, resource, action):
    grants = ROLE_PERMISSIONS.get(role, {})
    return ("*" in grants or action in grants.get(resource, set())
            or "*" in grants.get(resource, set()))
```

This expresses general role authority. Owner exclusions and financial/signatory checks still belong in the relevant routes.

```
# Concept expressed by services/event_availability.py
# Two half-open intervals conflict when:
new_start < existing_end and new_end > existing_start
```

This third snippet is an explanatory expression, not an additional project function. Adjacent intervals do not overlap; the real service also checks venue, active state and holds within a serialized transaction.

## 6.4 State transitions and repeated actions

Permit/event/blotter/schedule targets are validated against the acting role and current state. Payment unique keys and program claim keys enforce single records in the database. Queue/release/program, requirement review and verification/account conditional updates reject stale page decisions. For account approval, the SQL condition also requires both verification states still to be Verified. The newer stabilization suite exercises simultaneous account approvals and observes one approval log and one three-channel notification set. This is evidence for that tested race, not every possible concurrent action.

## 6.5 Compatibility names and inactive fields

`submission_receipt()` is a legacy internal name for the Submission Acknowledgment DOCX endpoint. `walkin_claim_stub()`/`claim-stub.docx` generate the renamed walk-in acknowledgment. `/announcements/<id>/delete` archives the row. Event attendee/attachment columns remain for historical data but are not current input features. Model existence alone does not prove a complete UI workflow: queue priority, referral assignment, blotter resolution and schedule notes must be assessed against routes/templates.

# Chapter 7 — Security and Privacy

## 7.1 Implemented controls

**Simple explanation.** Security checks who is signed in, what role they have, which record they own, and whether the action is valid now.

**Technical explanation.** Passwords are stored using Werkzeug hashes, not plaintext. Flask's signed session cookie identifies the account. HttpOnly and SameSite=Lax are configured; Secure depends on `SESSION_COOKIE_SECURE` and is enabled in the supplied Render configuration. The default development secret must be replaced outside local development. Protected routes recheck user activity/approval rather than trusting only hidden buttons.

CSRF tokens are random session values checked with constant-time comparison on POST/PUT/PATCH/DELETE using a form field or header. SQLAlchemy binds ordinary query values. Jinja's normal HTML escaping protects ordinary rendered variables; developers must preserve escaping and avoid introducing unsafe HTML/JavaScript sinks. These controls reduce specific risks; the system has not undergone a complete penetration test and does not implement every possible security defense.

Uploads accept PDF, PNG and JPEG extensions with matching file signatures and use opaque random filenames. The total request size is limited to 8 MiB, which includes multipart/form overhead rather than allowing 8 MiB for every file. Private paths are outside `static`; serving routes use basename resolution, existence checks and authorization. Signature uploads accept PNG/JPEG only. File signature checks are not malware scans or full document parsers.

Reset tokens are random, hashed, expiring and marked used; the production delivery channel is absent. Staff self-processing guards apply to transaction review and release. Program processing/payment recording prohibit the owner regardless of operational role. General assignment does not create a new permission grant.

## 7.2 Privacy access boundaries

**Table 9. 7.2 Privacy access boundaries**

| Data | Current access boundary | Documentation caution |
|---|---|---|
| Valid ID upload/number | Owner or Administrator document access; masked number in relevant views | No government verification API; ID changes restart review |
| Address/emergency/profile details | Own Profile; authorized operational/admin views | Do not export real profiles into documentation/screenshots |
| Permit/event/blotter records | Resident owner; Staff/Admin operational access | Assignment is not exclusive case access |
| Permit requirement files | Non-owner Staff processor or Administrator | Not a general Resident evidence download endpoint |
| Program application evidence | Owner or Staff/Admin; owner barred from processing | Aggregates exclude evidence and applicant narratives |
| Payment receipt | Owner or financially authorized personnel | Ordinary Staff financial access is limited |
| Household/concern/referral | Staff/Admin front desk | Sensitivity labels are not separate Staff clearance enforcement |
| Signature images | Private storage; configured Admin applies signature | Downloaded signed DOCX contains an image; not a cryptographic seal |
| Reports/logs | Administrator | Logs may contain operational details/IP; protect exported files |

## 7.3 Security and privacy limits

No database encryption, malware scanner, multi-factor login, comprehensive rate limiting, tamper-evident audit chain, retention/deletion schedule or external identity validation is implemented in the inspected source. This list describes the implementation, not a legal compliance assessment. Access by a host/database administrator remains outside the web application's permission boundary. Real deployment requires protected secrets, HTTPS, storage permissions, controlled backups and an approved operational policy.

## 7.4 Error handling

**Table 10. 7.4 Error handling**

| Condition/status | Actual behavior |
|---|---|
| Invalid login | Shows validation/authentication feedback without establishing an authorized session |
| Unauthenticated protected page | Login redirect; 401 handler also redirects to login |
| 400 | Invalid CSRF, malformed values or invalid transitions produce the configured bad-request response |
| 403 | Permission/ownership denial uses the configured forbidden response |
| 404 | Missing route/record uses the not-found response; some legacy document handlers redirect with a flash instead |
| 405 | Unsupported method uses the workflow error template |
| 409 | Duplicate/conflicting payment, release or program state uses conflict feedback |
| 413 | Oversized upload request uses the workflow error template |
| 422 | Registered workflow error response, when raised |
| 500 | Rolls back the session and renders the generic server-error page |
| Invalid upload/value | Common form handlers roll back and flash feedback or abort with 400 |
| Expired reset token | Redirects to Forgot Password with an invalid/expired message |

The application does not automatically retry every failed transaction. Operators should reload and inspect current state before repeating a financial or release decision. Error feedback should not be treated as proof that an external physical action did not occur.

# Chapter 8 — Reports and Analytics

## 8.1 Real-data reporting pipeline

**Simple explanation.** Reports count and summarize records already stored in SQLite. They do not invent sample values when no records exist.

**Technical explanation.** `validate_filters()` accepts report type and applicable date/year/month/status/source/service/Purok/program/category selections. `build_report()` executes scoped ORM aggregates and returns named sections containing real category/value rows. Browser Canvas charts and tables share these rows. `chart_png()` uses matplotlib for printable charts. `word_report()` writes native Word headings, paragraphs and tables plus chart images and source tables. `reports()`, `report_export()` and chart endpoints require Administrator permissions and audit relevant report actions.

## 8.2 Implemented reporting areas

**Table 11. 8.2 Implemented reporting areas**

| Report | Main data and interpretation |
|---|---|
| Overall Barangay | Combined supported module summaries |
| Residents | Registered/active approved profiles, residency/type/Purok distributions and registration trend |
| Households | Status counts and active-member household-size distribution |
| Permits | Status/service/source, approval/rejection/resubmission proportions, released count and measured duration |
| Events | Type/status/venue, submission trend, reserved hours and logged conflicts |
| Blotters | Case status and creation trend; no private narratives |
| General Concerns | Category/status and trend |
| Walk-in Transactions | Permit transactions whose source is not Online |
| Queue | Status/volume, recorded wait/serve duration where timestamps exist |
| Appointments | Schedule status and creation trend |
| Payments | Positive Paid collections, payment-time trend, service/event sums, waiver composition and unpaid requests |
| Scholarship | Program/application counts, states, claim rules and completed allocations |
| Ayuda | Same reusable reporting engine restricted to Ayuda |
| Announcements | Lifecycle/category and creation trend |
| Feedback | Response count, rating distribution and mean if ratings exist |
| Staff Operations | Recorded Staff actions and assisted transaction counts, not productivity rankings |
| Activity/Audit Summary | Action and actor-role counts |

Date filters normally apply to record creation. Payments use `paid_at`; unpaid counts explicitly use the submission period. Queue/date interpretation follows stored timestamps. Report time buckets use SQLite date functions with a +8-hour adjustment for stored UTC. Status filters represent current state, including on monthly composition charts; they are not a complete historical state-at-month-end reconstruction. Months without rows are not synthesized as observed transactions. Empty aggregates display no records rather than fake charts.

## 8.3 Visualization choices

Summary cards show compact totals. Lines show month-grouped trends; bars compare categories; donuts show composition for small category sets; stacked bars compare month/status composition; editable tables preserve exact values. Donuts with too many categories fall back to bars. A separate area-chart implementation is not present. The chart is an aid to reading counts, not a statistical forecast or causal analysis.

## 8.4 Word export and privacy

Administrator POST `/reports/export/docx` validates filters again, builds the report, creates a `reports` metadata row and records export actions. The DOCX includes the title, generating user/time, selected filters, summary, sections, charts and native editable tables. Chart pictures are not editable chart objects, but adjacent source tables remain editable. Resident ID images, private program evidence and blotter narratives are excluded from aggregates. The Staff Operations report can identify Staff by name; it is restricted administrative output.

All 17 report types produced structurally valid editable DOCX files in the test evidence. Opening the files in installed Microsoft Word was not verified. Application `/reports/export/pdf` is not implemented; this capstone PDF is a separate documentation deliverable.

# Chapter 9 — Testing and Quality Assurance

## 9.1 Evidence and methodology

Testing evidence comes from `tests/test_system.py`, `test_revision.py`, `test_stabilization.py`, `validate_sqlite.py`, `browser_audit.py`, `audit_inventory.py` and saved audit artifacts. This documentation does not turn source inspection into a fabricated runtime Pass. The current captured acceptance run passed **48 tests in 84.587 seconds (OK)**. The earlier October 5 audit recorded 35 passing tests; the latest run includes the newer stabilization cases. Exact captured output is retained in `documentation/acceptance-run.txt`.

Tests use isolated fixtures for account/role behavior, permits, requirement snapshots, payments, event reservations, programs, announcements, documents and database integrity. Concurrent tests exercise specific pairs of competing submissions. Browser tests used Edge at widths 1440, 1024, 768 and 390 with synthetic Administrator/Staff/Resident users. The saved result contains 160 page checks, zero captured JavaScript errors and zero horizontal-overflow detections. This is not a complete device/accessibility laboratory or load test.

## 9.2 Test cases and actual results

**Table 12. 9.2 Test cases and actual results**

| ID | Source scenario | Expected result | Actual / status |
|---|---|---|---|
| T01 | test_payment_authority_snapshot_duplicate_and_receipt | Only authorized recording; old fee preserved; duplicate blocked; Paid receipt valid. | Recorded OK |
| T02 | test_waiver_free_service_and_unpaid_release_guards | Admin waiver/free service handled; unpaid signing/release blocked. | Recorded OK |
| T03 | test_walkin_acknowledgment_print_ownership_and_atomic_requirements | Correct assisted document and owner access; incomplete requirements do not leave partial permit. | Recorded OK |
| T04 | test_announcement_boundaries_archive_and_timezone | Start/expiry boundaries, Manila conversion and archival preserve history. | Recorded OK |
| T05 | test_program_lifecycle_deadlines_privacy_and_release | Valid states/deadlines, protected evidence and authorized completion. | Recorded OK |
| T06 | test_household_duplicate_rule_and_repeatable_program | Household one-time claim blocked; repeatable program permits repeats. | Recorded OK |
| T07 | test_report_filters_totals_docx_and_privacy | Filters and totals match fixtures; private data omitted; DOCX readable. | Recorded OK |
| T08 | test_new_pages_all_roles_and_csrf | New page permissions and mutation token checks enforced. | Recorded OK |
| T09 | test_all_static_template_endpoints_exist | Referenced static url_for endpoint names resolve. | Recorded OK |
| T10 | test_full_permit_review_payment_signature_release_feedback | Complete verified-to-paid-to-signed-to-released lifecycle and one feedback. | Recorded OK |
| T11 | test_program_creation_publication_snapshot_replacement_and_slots | Draft/publication, snapshots, evidence replacement and capacity enforced. | Recorded OK |
| T12 | test_concurrent_payment_and_program_claim_are_unique | Competing submissions leave one payment and one one-time claim. | Recorded OK |
| T13 | test_concurrent_event_and_walkin_queue | Competing event creates one reservation; assisted tickets have distinct numbers. | Recorded OK |
| T14 | test_upload_type_size_path_and_pagination | Invalid/mismatched/oversized files rejected; safe names and page two work. | Recorded OK |
| T15 | test_word_exports_every_report_and_sqlite_integrity | All 17 DOCX report types valid; database integrity remains valid. | Recorded OK |
| T16 | test_legacy_announcement_migration_is_additive_and_idempotent | Legacy notices survive upgrade and repeated migration. | Recorded OK |
| T17 | test_uploaded_files_rollback_with_incomplete_request | Uncommitted private uploads removed after incomplete request. | Recorded OK |
| T18 | test_missing_verification_action_never_approves | Missing action does not verify ID or create an ID_VERIFIED log. | Recorded OK |
| T19 | test_private_id_preview_has_image_mimetype_and_no_cache | Authorized inline preview has correct MIME/no-store; unrelated resident denied. | Recorded OK |
| T20 | test_simultaneous_account_decisions_create_one_log_and_notice_set | Competing Admin approvals create one decision log and three notification-channel rows. | Recorded OK |
| T21 | test_rejection_is_final_and_preserves_metadata | Repeated decisions preserve rejection/time; final decision buttons absent. | Recorded OK |

| ID | Source scenario | Expected result | Actual / status |
|---|---|---|---|
| T22 | test_invalid_resident_update_can_be_rejected | Malformed proposed email can be rejected without changing stored email. | Recorded OK |
| T23 | test_profile_changes_invalidate_only_changed_verification | Address/ID changes reset verification while account approval stays Approved. | Recorded OK |
| T24 | test_front_desk_handles_service_without_configured_fee | Front desk renders successfully when a service fee is unset. | Recorded OK |
| T25 | test_admin_can_review_staff_address_correction | Administrator can reverify Staff profile after an address correction. | Recorded OK |
| T26 | test_repeated_requirement_review_does_not_create_history | Same-state repeated review returns 409 and leaves one history entry. | Recorded OK |
| T27 | test_missing_id_file_cannot_be_verified | Absent underlying ID file does not receive Verified status. | Recorded OK |
| T28 | test_dashboard_work_queue_keeps_older_actionable_requests | Older actionable requests remain visible despite newer Completed records. | Recorded OK |
| T29 | test_resident_dashboard_excludes_past_and_cancelled_appointments | Resident upcoming list omits past and Cancelled appointments. | Recorded OK |
| T30 | test_registration_is_pending_complete_and_gated_until_admin_approval | Registration stores fields; login gated until both verification and approval. | Recorded OK |
| T31 | test_rejected_registration_cannot_login_and_consent_is_required | Rejection denies login; privacy consent required. | Recorded OK |
| T32 | test_resident_type_and_residency_status_are_separate_and_others_require_details | Distinct profile fields and Others details enforced. | Recorded OK |
| T33 | test_id_document_is_private_and_account_approval_requires_both_verifications | Private ID boundary and dual-verification approval gate. | Recorded OK |
| T34 | test_final_account_decision_is_one_time | Repeated final decision preserves prior approval and does not duplicate decision log. | Recorded OK |
| T35 | test_configured_fees_are_snapshotted_and_unconfigured_services_are_blocked | Configured fees copied; unconfigured service submission blocked. | Recorded OK |
| T36 | test_blotter_respondent_is_required_saved_and_private | Respondent required/stored; unrelated Resident cannot see case. | Recorded OK |
| T37 | test_staff_can_submit_personal_permit_and_event_but_cannot_process_them | Personal submission succeeds; self-processing denied. | Recorded OK |
| T38 | test_overlapping_event_creates_private_ten_minute_hold_and_accepts_it | Overlap creates owner-private expiring recommendation that can be accepted. | Recorded OK |
| T39 | test_event_hold_decline_releases_slot | Declining recommendation makes held interval available. | Recorded OK |
| T40 | test_expired_event_hold_is_persistently_released | Expired hold no longer blocks and expiry persists. | Recorded OK |
| T41 | test_adjacent_event_intervals_do_not_overlap | Touching interval boundaries are allowed. | Recorded OK |
| T42 | test_acknowledgments_permit_document_and_reports_are_editable_docx_only | Native Word text/tables exist; application PDF export absent. | Recorded OK |

| ID | Source scenario | Expected result | Actual / status |
|---|---|---|---|
| T43 | test_schedule_creation_redirect_validation_and_record_ownership | Valid linked schedule created; malformed/foreign transaction rejected. | Recorded OK |
| T44 | test_report_uses_real_counts_and_exports_only_docx | Fixture aggregates match and export uses DOCX. | Recorded OK |
| T45 | test_role_boundaries_primary_pages_and_ten_second_toasts | Primary page role boundaries and configured toast behavior. | Recorded OK |
| T46 | test_sqlite_integrity_and_migration_are_idempotent | Repeated migration preserves data and SQLite integrity. | Recorded OK |
| T47 | test_password_reset_token_is_single_use_and_preserves_role | Consumed/expired token rejected; reset does not change role. | Recorded OK |
| T48 | test_front_desk_services_and_queue_transitions | Assisted intake and ordered queue state progression. | Recorded OK |
| B01 | Edge responsive audit | 160 pages, four viewport widths, three roles; no recorded JS/overflow issues | Pass in saved browser audit |
| DB01 | Isolated restart/integrity validator | Persistence across process restart, pragmas and DOCX checks | Pass in recorded SQLite validation |
| EXT01 | Real Word/printer/email/SMS | Actual external rendering/printing/delivery | Not tested / transport absent |

## 9.3 Performance observations

The isolated benchmark seeded 5,000 synthetic permit records. Post-change medians were 16.82 ms for Dashboard, 9.37 ms for front-desk search, 12.83 ms for Permit list, 3.32 ms for Event list, 3.58 ms for Activity Logs, 3.58 ms for Notifications, and 24.48 ms for Permit analytics. The measured Word export took 1,495.51 ms and returned 116,901 bytes. No comparable pre-change measurements exist, so no percentage improvement is claimed.

Pagination bounds many lists at 30 rows; announcements/program lists use their own 20/25-row sizes. Front-desk search is limited to 50. Query indexes support common owner/date filters. Some front-desk queues/ready lists and schedule choices still load all matching records. Reports execute several aggregate queries, and overall DOCX generation can be expensive. Busy buttons prevent ordinary repeated browser submissions, but database constraints and conditional writes remain necessary for concurrent requests.

## 9.4 Failed, partial and untested areas

The automated suite passed within its scenarios; warnings include datetime/SQLAlchemy deprecations and file-handle cleanup warnings. Real SMS/email delivery, physical printing, installed Microsoft Word rendering, production-scale multi-client load, a full penetration test and manual assistive-technology testing were not performed. There is no application PDF export to test. Structural validation of this documentation PDF is separate from application functionality.

Source inspection for documentation identified partial behaviors not covered by the earlier overall “stable within tested scope” statement: the pre-approval correction gap, appointment Rescheduled without time editing, unenforced queue priority, partial concern/referral and blotter completion forms, and broad operational access despite sensitivity labels. The current working tree also contains newer verification reset/conditional-update fixes, which this document incorporates. These distinctions prevent the earlier audit summary becoming a blanket assurance.

# Chapter 10 — User Manuals

## 10.1 Resident manual

1. Open Register. Complete account, personal, residence and emergency-contact fields. Choose an active accepted ID type, upload supported evidence and provide privacy consent. Submit once.

2. Wait for Administrator verification and approval. Pending/rejected accounts cannot use normal service pages. If correction is required before approval, contact the office; a complete unauthenticated correction portal is not implemented.

3. Log in after approval. Check the Dashboard, Announcements and Notifications. Use Manage Profile to change permitted information or your password. ID changes restart ID review; street/Purok changes restart address review.

4. Open My Permits, select an available service and review its configured fee/requirements. Enter purpose and all required evidence. Save the Submission Acknowledgment and reference.

5. Open the request detail to track evidence and processing states. If an item is Needs Resubmission, read the reason and use its replacement input. Resubmit the correct file/text; do not assume replacing one item approves the whole request.

6. After final approval, follow the office's payment instructions if a positive fee applies. Only authorized personnel record payment. Use Payments to view your state and download a receipt only when a Paid record exists.

7. When the permit is signed and Ready for Pickup, download the approved DOCX and follow release instructions. At the counter, personnel confirm identity and record release. Downloading alone does not record counter release.

8. Open My Event Requests, choose category/venue, enter event details and a valid date/time interval. If a conflict is shown, accept the held recommendation before expiry or decline and choose another date. Keep the acknowledgment.

9. Open My Blotter Reports and enter respondent, incident and narrative details accurately. Track the case state; the software does not provide legal advice or automatically schedule a hearing from a state label.

10. Open My Schedules and choose one of your existing transactions. Request a date/time and purpose, then monitor its state. Residents do not confirm/cancel schedules through operational status controls.

11. Read Notifications and mark messages read. Email/SMS labels describe simulated records in this version.

12. Open Community Programs, read eligibility/dates/instructions and submit required evidence. One-time Resident/Household claim rules apply. Follow Needs Requirements feedback and monitor review/completion.

13. After your permit is Completed, submit one 1–5 feedback rating and an optional comment. Repeated ratings for the same permit are rejected.

## 10.2 Barangay Staff manual

1. Log in using your assigned active account. Review operational summaries and use Resident Services for assisted work.

2. Search a resident by name, numeric ID, contact, household or Purok. Confirm the correct person before selecting Assist this Resident. Search cards are not independent identity proof.

3. Select a configured permit service, enter purpose/counter and capture all required evidence. Submit to create the normal permit record and a queue ticket. Print/download the Walk-in Transaction Acknowledgment.

4. Progress the ticket through Now Serving, Start Service and Complete Service. Queue completion is separate from permit approval/release. Apply any locally approved priority policy manually because no automatic priority engine is present.

5. Open a permit not owned by you. Inspect required files/text and set appropriate review states. Give a specific correction reason when returning evidence. Endorse only after all required items are verified.

6. Review/endorse eligible event requests and progress blotter states within your permissions. Final permit/event approval is Administrator work. An event schedule conflict is resolved through the reservation flow, not by changing an unrelated appointment.

7. Create/confirm/complete transaction-linked appointments as permitted. The Rescheduled control currently changes status only; coordinate the actual correction with the Administrator rather than assuming it edits time.

8. Assign permits to an eligible colleague and add a useful note. Assignment does not permit a colleague to process their own request or gain Admin authority.

9. At release, inspect the ready permit, confirm the resident's identity and record release. The backend also checks signature and payment. Do not release your own request.

10. Record general concerns and route them; submit proposed profile changes; create/transfer household membership; record referrals. Recognize that concern/referral resolution and move-out workflows are incomplete.

11. Draft Scholarship/Ayuda programs or announcements, assist approved applicants, review evidence and qualify applications. Administrator approval/publication/completion controls remain separate.

12. Record manual payments only after explicit Administrator authorization. You cannot waive fees or record your own payment. Ordinary Staff accounts show their own financial transactions only.

13. Submit personal Resident requests using your account, then let another authorized person process them. No second account is required merely to separate personal requests from duties.

## 10.3 Administrator manual

1. Review Dashboard totals and pending decisions. Open User Management to inspect registration details and private ID evidence.

2. Verify or reject each Pending ID/address review. Supply a correction reason when rejecting. Approve the Pending account only after both verifications are Verified. Sequential repeat decisions are rejected.

3. Manage account roles/activity carefully; preserve an active Administrator. Configure active Resident Types and accepted ID types according to approved office policy.

4. In Configuration, create/update Permit Types, Event Categories and Venues. Explicitly configure each service fee, including zero for free services. Unconfigured pricing makes new service submission unavailable.

5. Add Document/Text/System Check requirements with required/optional flags and order. Changes apply to new applications; historical snapshots remain attached to older applications.

6. Configure each permit's protected signature image, name/title and authorized Administrator. Complete final review after endorsement. For paid services, record payment or a justified waiver before signing.

7. In Payments, grant/revoke Staff authority as needed. Record only verified physical cash collection at the exact fee snapshot. Use a waiver only with a reason. Download a receipt only for an actual Paid record.

8. Apply the signature as the designated Administrator, then allow release through the counter. Approval, signing and release are distinct actions.

9. Review Staff program drafts, confirm eligibility/requirements/deadlines/claim rule/slots and publish. Review Qualified applications, approve/waitlist within capacity, and confirm assistance release or scholarship completion when it actually occurs.

10. Approve announcement drafts with valid future expiry. Use start/expiry times in Asia/Manila. Archive obsolete notices; expired history is retained.

11. Use Reports & Analytics to choose one of 17 report types and only applicable filters. Read exact tables alongside charts. Export DOCX; protect the downloaded administrative report.

12. Use Activity Logs to trace recorded actions. Logs support accountability but are not a comprehensive, tamper-proof record of every database operation.

13. Review proposed resident updates and household consistency. Approved address/Purok edits restart address verification; inspect the new information before verifying it again.

# Chapter 11 — Deployment, Maintenance and Backup

## 11.1 Environment and dependencies

The local verification environment uses Python 3.14. The supplied Render configuration specifies Python 3.13.15; source uses `datetime.UTC`, so older runtimes must be assessed before use. Exact application dependencies are pinned in `barangay_system/requirements.txt`. Gunicorn is for the supplied Linux deployment; `run.ps1` uses Flask's development server for local work. The initial supported-ID catalog is not automatically accepted for registration: the Administrator must activate acceptance according to local policy.

From the project root on Windows:

```
.\setup.ps1
# Initialize catalogs/schema, and create the first Administrator
# only after securely supplying INITIAL_ADMIN_* variables.
cd barangay_system
..\.venv\Scripts\python.exe seed.py
cd ..
.\run.ps1
```

Before first `seed.py`, set `INITIAL_ADMIN_USERNAME`, `INITIAL_ADMIN_EMAIL`, `INITIAL_ADMIN_NAME` and a privately chosen `INITIAL_ADMIN_PASSWORD` through your deployment environment. This documentation deliberately provides no reusable password. The seeder preserves an existing Administrator and creates missing base permit/event catalogs; it does not create demonstration Staff/Resident accounts. New service prices are not automatically approved merely by seeding. `flask --app app init-db` is available for additive schema initialization; `create-admin` is also exposed as a CLI command in `app.py`.

## 11.2 Configuration reference

**Table 13. 11.2 Configuration reference**

| Variable/setting | Purpose | Operational note |
|---|---|---|
| SECRET_KEY | Signs sessions | Set a strong private value; do not use development fallback |
| DATA_DIR | Base persistent runtime directory | Contains private files and default database |
| SQLITE_DB_PATH | SQLite file path | Relative paths resolve against the application directory |
| DATABASE_URL | Optional SQLite connection override | Non-SQLite URL rejected; coordinate with backup path |
| SESSION_COOKIE_SECURE | HTTPS-only session cookie | Enable under HTTPS |
| FLASK_DEBUG | Development debug mode | Disable for public deployment |
| SMS_PROVIDER | Configuration placeholder | Does not install or activate an actual SMS integration |
| BARANGAY_NAME / CITY_MUNICIPALITY | Configured naming values | Some document calls also contain fixed project labels |
| INITIAL_ADMIN_* | Initial account provisioning | Needed when no Admin exists; keep password private |
| RESET_TOKEN_MINUTES | App-config default: 30 | Set in app configuration; not automatically an environment parser |

Keep `DATABASE_URL` and `SQLITE_DB_PATH` consistent: the runtime may use the former while `backup_sqlite.py` reads the latter. The backup utility does not automatically derive its source from an overridden SQLAlchemy URL.

## 11.3 Deployment boundary

`render.yaml` starts `python seed.py` then Gunicorn with one worker, four threads and a 120-second timeout, and uses `/healthz`. Its configured `/tmp/minante-data` path is demonstration storage; the project deployment notes identify the ephemeral-storage limitation. A real persistent installation needs a persistent volume, one service instance, protected secrets and controlled upgrades. This documentation does not claim a successful production deployment or inspect the remote GitHub repository.

## 11.4 SQLite backup and recovery

**Simple explanation.** A usable backup includes both the transaction database and the documents it refers to. Restoring only the database can leave missing IDs/evidence/signatures.

**Technical explanation.** From the application directory, run `..\.venv\Scripts\python.exe backup_sqlite.py` with the same configuration as the service. The utility opens the configured SQLite file, uses sqlite3's backup API to a timestamped `.partial` file, runs `PRAGMA integrity_check`, then renames the verified file into `DATA_DIR/backups`. This is safer for a live WAL database than copying only the main database file during writes.

For a coordinated recovery point, pause application writes, create the database backup, and copy the matching private uploads, identifications, signatures and relevant generated files into controlled backup storage. Record configuration and software version without publishing secrets. Resume only after the backup set is confirmed. The provided utility backs up the database, not the private directories or an off-host archive.

To restore, stop the service, preserve the current data set as a rollback copy, restore a chosen database/private-file set into a separate test location, check integrity and representative document access, then switch the configured paths and restart one service instance. Verify account access, key record counts and authorized document downloads before reopening use. Do not overwrite a live database with active WAL writers. Backup restoration drills and retention/encryption schedules are recommended operational procedures, not automated features claimed in code.

## 11.5 Maintenance and handover

1. Record the current source version and back up before schema upgrades.

2. Run additive initialization, then appropriate acceptance tests on an isolated copy.

3. Review application errors, operational logs, disk space and failed/simulated notification states.

4. Add new Permit Types and requirements through Configuration; explicitly set fees and signatory authority before offering them.

5. Explain to personnel that fee/requirement changes affect new submissions, not old snapshots.

6. Create programs with clear dates/claim rules and publish only after review. Check expired announcements in history rather than deleting records.

7. Update pinned dependencies deliberately and retest affected behavior; current deprecation/resource warnings deserve maintenance attention.

8. Reconcile private-file storage, household membership and historical financial flags where needed. Do not fabricate missing financial history.

## 11.6 GitHub and documentation maintenance

The intended repository is `tagalfranz23/My-projects-`. This package is based on the available local working tree, not a remotely verified commit. Git was unavailable during the preceding audit; tracked-file status and remote contents were not certified.

Root/application ignore rules cover `.env`, databases/WAL files, runtime uploads, private IDs/signatures, backups, Python caches and generated audit artifacts. Ignore rules do not remove files already committed. Before

publishing, the maintainer must inspect the actual staged diff and tracked files. Documentation must never contain real resident rows, ID images, credentials, private program documents or live signature assets.

`documentation/CAPSTONE_DOCUMENTATION.md` is the current narrative source; `source_evidence.json` captures code/schema metadata without operational rows. `build_documentation.py` generates the editable DOCX, PDF, cheat sheet and validation summary. `documentation/requirements.txt` keeps PDF build tools separate from application runtime requirements. Historical research files are retained and are not authoritative for current behavior.

# Chapter 12 — Strengths, Limitations and Future Work

## 12.1 Verified strengths

The source provides centralized related records; clear Resident/Staff/Admin responsibilities; server-side ownership controls; fee/requirement snapshots; a common online/assisted permit workflow; tested reservation/payment/claim constraints; private upload paths; reusable community programs; time-bounded announcement feeds; and real aggregate reporting with editable Word output. These are design and implementation strengths, not claims of perfect usability or measured community impact.

## 12.2 Current limitations and weak areas

**Table 14. 12.2 Current limitations and weak areas**

| Area | Current limitation | Practical implication |
|---|---|---|
| Notifications/recovery | No live email/SMS transport | Production password recovery delivery and external reminders need integration |
| Verification | Address/ID reset is implemented, but a pending/rejected correction portal is incomplete | Reset requests human review; it is not automatic verification |
| Concurrency | Conditional/unique protection is not used uniformly | Sequential guards do not prove all competing decisions execute once |
| Scheduling/time | Rescheduled does not edit time; older event/schedule naive times differ from explicit Manila conversion | Appointment/timezone behavior needs focused work |
| Priority/assignment | Stored priority is not applied; office/officer scheduling is incomplete | Operational policy remains manual |
| Blotter/concern/referral | Several stored fields and closing workflows lack complete forms | State/intake support is narrower than full case management |
| Household data | Direct profile text can diverge from actual memberships | Use verified membership rows for household claims |
| Walk-in eligibility | Helper does not universally enforce approved active profile | Personnel must verify assisted-client eligibility |
| Financial scope | Cash exact amount/waiver only; no refund or reconciliation workflow | Not a complete accounting/payment-gateway system |
| Signatures | Editable document with signature image | No certificate-based integrity/nonrepudiation guarantee |
| Storage | Local private files; unencrypted SQLite; backup utility is database-only | Host security and coordinated file backups are necessary |
| Scale | One SQLite writer; several unbounded operational lists | Real-load testing required before expanding use |
| Audit/privacy | Explicit logs only; sensitivity labels do not add fine-grained Staff clearance | No universal change capture or case-assignment privacy guarantee |
| Testing | Synthetic scenarios, no installed Word/printer/provider verification | Do not present automated Pass as all-environment certification |

## 12.3 Future enhancements — not current features

Future work should first complete pre-approval correction, appointment time edits and timezone consistency, case/referral completion forms, household move-out reconciliation, eligibility checks and conditional state updates where missing. Separate, later enhancements may include approved SMS/email transport, payment reconciliation or a gateway, cryptographic document signing, malware scanning, encrypted off-host backups and restore automation, multi-factor/rate-limit protections, fine-grained case access and broader accessibility/load testing. A mobile client or different database should follow demonstrated needs rather than being described as already implemented.

# Chapter 13 — Capstone Defense Reviewer

The following questions and answers refer to this inspected implementation. They are preparation material, not a record of questions actually asked by a defense panel.

## Purpose and research

**Q01. What is the system?**

A Flask application that centralizes resident accounts and Barangay service transactions with review, scheduling, documents and monitoring.

**Q02. What is the complete capstone title?**

Development of an Integrated Barangay Permit, Event Approval, Blotter Management System with Notification and Scheduling Features for Barangay Minante 1, Cauayan City.

**Q03. What problem does it address?**

It is designed to reduce fragmented records and improve transaction tracking and coordination; no measured local baseline is invented.

**Q04. What is the general objective?**

To digitize and centralize service processing, monitoring, scheduling, communication records and administration.

**Q05. Does it replace Barangay officials?**

No. It records and supports decisions made by authorized personnel.

**Q06. Who benefits?**

Residents, Staff, Administrators and future maintainers benefit from clearer records and workflows; impact still needs real evaluation.

**Q07. Were survey results collected for this document?**

No survey evidence was supplied; the document uses source inspection and actual test artifacts.

**Q08. Why include limitations in a defense?**

They distinguish verified implementation from intended improvements and keep claims defensible.

## Users and permissions

**Q09. Which roles exist?**

Resident, Staff and Administrator.

**Q10. Why separate Staff from Administrator?**

Staff handle intake/review/endorsement while Administrators control final decisions and configuration.

**Q11. Can Staff submit personal requests?**

Yes, but they cannot process their own permit/event/blotter requests through operational review.

**Q12. Does assignment give additional authority?**

No. Role, ownership, workflow and specific authorization checks still apply.

**Q13. Can ordinary Staff record payments?**

Only after an Administrator grants explicit payment authority; Staff cannot waive fees.

**Q14. Can Residents view another resident’s request?**

Owner checks reject that access on protected resident transaction routes.

**Q15. Who sees all reports?**

The Administrator; system reports and chart/export routes are not general Staff/Resident features.

**Q16. Can any Admin sign any permit?**

Only the configured authorized Administrator for that permit type can apply its signature.

## Accounts and security

**Q17. What is authentication?**

Checking credentials and account state to establish who is signed in.

**Q18. What is authorization?**

Checking whether that signed-in account may perform a specific action on a record.

**Q19. How are passwords stored?**

As Werkzeug-generated hashes, not plaintext passwords.

**Q20. Is Flask-Login used?**

No. The app has custom session loading and login_required logic.

**Q21. What prevents CSRF?**

A random session token is validated on mutating HTTP requests.

**Q22. Why are ID and address checks separate?**

Identity evidence and residence information are different facts reviewed before registration approval.

**Q23. What prevents repeat account decisions?**

The current handler requires Pending state and uses a conditional SQL update; approval also requires both verification states.

**Q24. What happens when ID/address information changes?**

Tracked ID type/number/file or street/Purok changes reset the relevant verification to Pending Verification.

**Q25. How are IDs protected?**

They are stored outside static files and served to their owner or Administrator through an authorized route.

**Q26. How does password recovery work?**

A random token is stored as a digest with expiry and consumption state; actual recovery email delivery is not integrated.

**Q27. Is upload validation a malware scanner?**

No. It checks allowed extension, signature, size and private storage, not malware behavior.

**Q28. Are logs tamper-proof?**

No. They record explicit actions but are ordinary database rows, not a cryptographic audit chain.

## Database and programming

**Q29. Why SQLite?**

It fits this configured single-service deployment with persistent relational storage and no separate database server.

**Q30. Is MariaDB the current database?**

No. Runtime configuration is SQLite-only; older mentions are historical.

**Q31. How does Flask access the database?**

Flask-SQLAlchemy maps models and sends bound SQL through the sqlite3 driver.

**Q32. What is a primary key?**

A unique row identifier, such as users.id.

**Q33. What is a foreign key?**

A relationship link, such as permit_applications.applicant_id referencing users.id.

**Q34. What is a transaction?**

A group of related changes committed together or rolled back on failure.

**Q35. Does flush mean commit?**

No. Flush sends pending SQL and obtains IDs while keeping the transaction open.

**Q36. What happens on incomplete permit requirements?**

Validation rejects submission; database rollback and staged-upload cleanup avoid partial records/files.

**Q37. How many tables were inspected?**

35 application tables in the local SQLite schema.

**Q38. Are all relationships physical foreign keys?**

No. Schedule/notification related_type and related_id are logical links checked by routes.

**Q39. Which programming languages are used?**

Python and JavaScript, with HTML/CSS, Jinja templates and SQL for presentation/data access.

**Q40. Is this a microservices application?**

No. It is a server-rendered Flask monolith with supporting services and directly registered routes.

## Permits and events

**Q41. How do dynamic requirements work?**

Admin definitions are copied into each permit’s requirement records at submission.

**Q42. Why snapshot requirements?**

Later catalog edits should not change evidence expected for an existing application.

**Q43. What does System Check validate?**

The implemented shared check confirms an Approved resident profile; it is not an arbitrary external policy engine.

**Q44. When can Staff endorse a permit?**

After all required requirement snapshots are Verified and the role/state transition is allowed.

**Q45. What is the difference between acknowledgment and approved permit?**

Acknowledgment records receipt; the approved document requires later approval, payment eligibility and signature.

**Q46. Why snapshot fees?**

A new fee must not rewrite the amount attached to earlier requests.

**Q47. Is the signature cryptographic?**

No. It is a private image applied by a designated Admin with stored signer/time metadata.

**Q48. How are double bookings prevented?**

A venue write serializes conflict checks before reserving overlapping intervals in SQLite.

**Q49. Do adjacent event intervals conflict?**

No. An event ending at 10:00 and another starting at 10:00 are adjacent.

**Q50. How long does a recommended slot stay held?**

The default private hold is ten minutes.

**Q51. What releases a reserved slot?**

Declining/expiring a hold or changing an event to a supported Rejected/Cancelled state releases it.

**Q52. Does the event form still require expected attendees?**

No. That is a legacy stored column, not an active event input.

## Front desk and payments

**Q53. How are residents without internet served?**

Staff/Admin encode a permit for an existing resident profile, collect requirements and generate a queue/acknowledgment.

**Q54. Who owns a walk-in permit?**

The Resident remains the applicant; the assisting employee is separately recorded as encoder.

**Q55. Is a walk-in acknowledgment a payment receipt?**

No. A receipt requires a real positive Paid row.

**Q56. How are queue duplicates prevented?**

A daily category sequence is combined with a unique queue_date/queue_number constraint.

**Q57. Is priority assistance automatic?**

No. A priority column exists, but automated ranking and category capture are not implemented.

**Q58. What prevents duplicate release?**

A conditional update requires Ready for Pickup before setting Completed and recording release.

**Q59. What is checked before release?**

Ready/signed state, payment eligibility, operator authority and an identity-confirmation checkbox.

**Q60. Does every service require a payment?**

No. A configured zero fee produces Not Required.

**Q61. Can a payment differ from the snapshot fee?**

The current Paid flow requires the exact fee; only an Admin waiver records zero with a reason.

**Q62. Does the system accept online payments?**

No. It records manual Cash payments and authorized waivers.

## Programs and announcements

**Q63. Are Scholarship and Ayuda hardcoded individual campaigns?**

No. They are program types in a configurable shared program model.

**Q64. Who determines eligibility?**

Personnel review the configured criteria and evidence; the software does not automatically award benefits.

**Q65. How are duplicate one-time claims blocked?**

A unique program/claim_key pair identifies the resident or active household claim.

**Q66. What does Repeatable mean?**

A null claim key permits repeated applications rather than a one-time resident/household claim.

**Q67. How are limited slots enforced?**

Admin approval counts Approved/Completed allocations under a SQLite writer lock before granting a slot.

**Q68. Does program completion transfer money?**

No. It records authorized release/completion confirmation and actor/time.

**Q69. How do announcements publish and expire?**

Time-filtered queries enforce visibility; backend access persists boundary status changes and logs.

**Q70. Is a background scheduler running?**

No. Persisted lifecycle updates occur on backend access, while feed filters enforce time boundaries.

**Q71. Are expired announcements deleted?**

No. They remain in history; archival also preserves the row.

**Q72. Which timezone is explicit for programs/notices?**

Asia/Manila inputs are converted to UTC for storage.

## Reports, testing and limits

**Q73. Do reports use fake sample charts?**

No. They aggregate SQLite records; empty data is not filled with invented transactions.

**Q74. Which charts are implemented?**

Lines, bars, donuts and stacked bars, with exact tables; a separate area chart is not implemented.

**Q75. Why are configured fees not collections?**

A fee is an amount due; only recorded positive Paid amounts are counted as actual collections.

**Q76. Are Word exports editable?**

Headings, paragraphs and tables are native Word content; charts are pictures with editable source tables.

**Q77. Is Chart.js used?**

No. Browser charts use custom Canvas code; printable charts use matplotlib.

**Q78. What do the tests prove?**

They establish behavior for the executed fixture scenarios, not that every future input or deployment is defect-free.

**Q79. Were real SMS, printing and Word rendering tested?**

No. Provider delivery, hardware printing and installed Microsoft Word rendering remain unverified.

**Q80. What is incomplete in appointments?**

The Rescheduled action changes status without editing the stored scheduled time.

**Q81. How should backups be taken?**

Use the SQLite backup API utility and coordinate matching private-file backups; test restoration separately.

**Q82. What should be improved first?**

Complete correction and partial workflows, reconcile time handling, strengthen remaining concurrent transitions, and validate production operations.

# Glossary

**Table 15. Glossary**

| Term | Meaning in this project |
|---|---|
| API | An interface for software requests; this app mainly provides HTML form routes rather than a public REST service. |
| Activity Log | An explicit record of actor, action, target and time. |
| Authentication | Establishing the account identity and allowed login state. |
| Authorization | Checking whether an account may perform an action. |
| Backend | Python routes, services, validation and persistence logic. |
| CRUD | Create, Read, Update and Delete operations on records. |
| CSRF | A forged cross-site mutation request; session tokens help reject it. |
| Commit | Make a successful database transaction durable. |
| DFD | Data Flow Diagram describing processes, actors and stores. |
| Database | Organized persistent data and relationships. |
| Electronic Signature | A recorded signature image/authorization in this implementation; not cryptographic signing. |
| Flask | The Python web framework used by this application. |
| Foreign Key | A column linking a row to a referenced table key. |
| Frontend | Browser-rendered HTML/CSS and JavaScript interaction. |
| IPO | Input–Process–Output model of system behavior. |
| Index | A database structure supporting selected lookups/orderings. |
| Jinja | The server-side template language rendering HTML pages. |
| Primary Key | The unique identifying key of a database row. |
| RBAC | Role-Based Access Control: permissions grouped by account role. |
| Requirement Snapshot | Copied requirements/evidence attached to an application rather than live catalog definitions. |
| Rollback | Discard pending database changes after a failed transaction. |
| Route | A URL/method mapped to a Flask handler. |
| SQL | Structured Query Language for relational data operations. |
| SQLite | Embedded relational database stored in a local file. |
| Session | Signed browser-held session data used to identify a login; not the password itself. |
| Transaction | A unit of related database changes committed or rolled back together. |
| Verification | A recorded human or implemented system check of evidence; not automatic government authentication. |
| WAL | Write-ahead logging, the configured SQLite journal mode. |

| Term | Meaning in this project |
| --- | --- |
| Waiver | An Administrator decision setting a payable request to Waived with zero collection and a reason. |
| Walk-in Request | An office-assisted submission owned by the resident and encoded by personnel. |

# Appendix A — Complete SQLite Data Dictionary

The following tables come from a read-only inspection of the local SQLite schema. All actual columns, declared types, primary keys, defaults, foreign keys and indexes are listed. No operational row values are included. SQLite primary keys are treated as identifying fields even where `PRAGMA table_info` reports the not-null flag as zero. Nullability and DDL are storage facts, not a replacement for route validation.

## A.1 accepted_id_types

ID catalog with active and accepted-for-registration flags.

**Created/updated by:** Migration/Administrator configuration.

**Table 16. A.1 accepted_id_types**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| name | VARCHAR(120) | — | No | — |
| description | TEXT | — | Yes | — |
| is_active | BOOLEAN | — | No | — |
| is_accepted_for_registration | BOOLEAN | — | No | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** name.

**Lookup indexes:** ix_accepted_id_types_is_active (is_active); ix_accepted_id_types_is_accepted_for_registration (is_accepted_for_registration).

## A.2 activity_logs

Explicit actor/action/resource/reference audit records.

**Created/updated by:** Instrumented handlers; Admin reads; no ordinary delete route.

**Table 17. A.2 activity_logs**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| user_id | INTEGER | FK → users.id | Yes | — |
| actor_role | VARCHAR(20) | — | Yes | — |
| action | VARCHAR(120) | — | No | — |
| resource | VARCHAR(50) | — | No | — |
| target_reference | VARCHAR(80) | — | Yes | — |
| details | TEXT | — | Yes | — |
| ip_address | VARCHAR(50) | — | Yes | — |
| user_agent | VARCHAR(255) | — | Yes | — |
| timestamp | DATETIME | — | No | — |

**Unique index keys:** No additional unique index listed beyond the primary key.

**Lookup indexes:** ix_activity_logs_action (action); ix_activity_logs_resource (resource); ix_activity_logs_user_id (user_id); ix_activity_logs_timestamp (timestamp).

**Relationships:** user_id → users.id (on delete NO ACTION).

## A.3 announcements

Dated notices, categories, approval and optional linked program.

**Created/updated by:** Staff draft/Admin publication; lifecycle refresh/archive.

**Table 18. A.3 announcements**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| title | VARCHAR(180) | — | No | — |
| body | TEXT | — | No | — |
| is_published | BOOLEAN | — | No | — |
| status | VARCHAR(20) | — | No | — |
| approved_by | INTEGER | FK → users.id | Yes | — |
| approved_at | DATETIME | — | Yes | — |
| published_at | DATETIME | — | No | — |
| expires_at | DATETIME | — | Yes | — |
| created_by | INTEGER | FK → users.id | No | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |
| category | VARCHAR(60) | — | No | 'General Advisory' |
| publish_start_at | DATETIME | — | Yes | — |
| expired_at | DATETIME | — | Yes | — |
| program_id | INTEGER | FK → community_programs.id | Yes | — |

**Unique index keys:** No additional unique index listed beyond the primary key.

**Lookup indexes:** ix_announcements_publish_start_at (publish_start_at); ix_announcements_published_at (published_at); ix_announcements_is_published (is_published); ix_announcements_created_by (created_by); ix_announcements_status (status).

**Relationships:** created_by → users.id (on delete NO ACTION); approved_by → users.id (on delete NO ACTION); program_id → community_programs.id (on delete NO ACTION).

## A.4 application_requirements

Copied requirement definitions and current submitted evidence.

**Created/updated by:** Submission snapshot; eligible owner resubmission and personnel review.

**Table 19. A.4 application_requirements**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| application_id | INTEGER | FK → permit_applications.id | No | — |
| definition_id | INTEGER | FK → service_requirements.id | No | — |
| name | VARCHAR(180) | — | No | — |
| requirement_type | VARCHAR(30) | — | No | — |
| instructions | TEXT | — | Yes | — |
| is_required | BOOLEAN | — | No | — |
| status | VARCHAR(30) | — | No | — |
| value | TEXT | — | Yes | — |
| file_path | VARCHAR(255) | — | Yes | — |
| submitted_at | DATETIME | — | Yes | — |
| reviewed_by | INTEGER | FK → users.id | Yes | — |
| reviewed_at | DATETIME | — | Yes | — |
| review_note | TEXT | — | Yes | — |

**Unique index keys:** application_id, definition_id.

**Lookup indexes:** ix_application_requirements_reviewed_by (reviewed_by); ix_application_requirements_status (status); ix_application_requirements_application_id (application_id).

**Relationships:** reviewed_by → users.id (on delete NO ACTION); definition_id → service_requirements.id (on delete NO ACTION); application_id → permit_applications.id (on delete NO ACTION).

## A.5 assistance_referrals

Assistance referral intake with sensitivity/status fields.

**Created/updated by:** Staff/Admin intake; later assignment/closure not exposed.

**Table 20. A.5 assistance_referrals**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| reference_no | VARCHAR(40) | — | No | — |
| resident_id | INTEGER | FK → users.id | No | — |
| category | VARCHAR(120) | — | No | — |
| description | TEXT | — | No | — |
| assigned_to | INTEGER | FK → users.id | Yes | — |
| status | VARCHAR(30) | — | No | — |
| sensitivity_level | VARCHAR(20) | — | No | — |
| created_by | INTEGER | FK → users.id | No | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** reference_no.

**Lookup indexes:** ix_assistance_referrals_resident_id (resident_id); ix_assistance_referrals_status (status).

**Relationships:** created_by → users.id (on delete NO ACTION); assigned_to → users.id (on delete NO ACTION); resident_id → users.id (on delete NO ACTION).

## A.6 blotter_cases

Complainant/respondent and incident record with case state.

**Created/updated by:** Resident/Staff/Admin intake; authorized status processing.

**Table 21. A.6 blotter_cases**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| case_no | VARCHAR(40) | — | No | — |
| complainant_id | INTEGER | FK → users.id | No | — |
| respondent_name | VARCHAR(150) | — | No | — |
| respondent_address | VARCHAR(255) | — | Yes | — |
| incident_type | VARCHAR(120) | — | No | — |
| incident_date | DATETIME | — | No | — |
| incident_location | VARCHAR(255) | — | No | — |
| narrative | TEXT | — | No | — |
| supporting_document_path | VARCHAR(255) | — | Yes | — |
| status | VARCHAR(30) | — | No | — |
| filed_date | DATETIME | — | Yes | — |
| assigned_officer | VARCHAR(150) | — | Yes | — |
| resolution | TEXT | — | Yes | — |
| hearing_date | DATETIME | — | Yes | — |
| updated_by | INTEGER | FK → users.id | Yes | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** case_no.

**Lookup indexes:** ix_blotter_owner_filed (complainant_id, filed_date); ix_blotter_cases_status (status); ix_blotter_cases_complainant_id (complainant_id).

**Relationships:** updated_by → users.id (on delete NO ACTION); complainant_id → users.id (on delete NO ACTION).

## A.7 community_programs

Scholarship/Ayuda definitions, dates, claim rules and requirements.

**Created/updated by:** Staff/Admin draft; Admin publish/requirements/close.

**Table 22. A.7 community_programs**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| program_type | VARCHAR(30) | — | No | — |
| title | VARCHAR(180) | — | No | — |
| description | TEXT | — | No | — |
| provider | VARCHAR(180) | — | Yes | — |
| assistance_type | VARCHAR(120) | — | Yes | — |
| eligibility | TEXT | — | No | — |
| target_categories | TEXT | — | Yes | — |
| instructions | TEXT | — | Yes | — |
| requirements_json | TEXT | — | No | — |
| application_start | DATETIME | — | No | — |
| application_deadline | DATETIME | — | No | — |
| distribution_date | DATETIME | — | Yes | — |
| slots | INTEGER | — | Yes | — |
| claim_rule | VARCHAR(30) | — | No | — |
| applications_enabled | BOOLEAN | — | No | — |
| status | VARCHAR(30) | — | No | — |
| created_by | INTEGER | FK → users.id | No | — |
| approved_by | INTEGER | FK → users.id | Yes | — |
| published_at | DATETIME | — | Yes | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Stored CHECK clauses:** program_type IN ('Scholarship', 'Ayuda'); claim_rule IN ('Per Resident', 'Per Household', 'Repeatable'); slots IS NULL OR slots > 0.

**Unique index keys:** No additional unique index listed beyond the primary key.

**Lookup indexes:** ix_community_programs_status (status); ix_community_programs_application_start (application_start); ix_community_programs_application_deadline (application_deadline).

**Relationships:** approved_by → users.id (on delete NO ACTION); created_by → users.id (on delete NO ACTION).

## A.8 event_categories

Event category and configured fee catalog.

**Created/updated by:** Seeder/Administrator configuration.

**Table 23. A.8 event_categories**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| name | VARCHAR(120) | — | No | — |
| description | TEXT | — | Yes | — |
| fee | NUMERIC(10, 2) | — | Yes | — |
| fee_is_configured | BOOLEAN | — | No | — |
| is_active | BOOLEAN | — | No | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** name.

**Lookup indexes:** No separate lookup index listed.

## A.9 event_requests

Event applicant, interval, category/venue, fee and decisions.

**Created/updated by:** Resident/Staff submission; non-owner Staff/Admin processing.

**Table 24. A.9 event_requests**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| reference_no | VARCHAR(40) | — | No | — |
| requester_id | INTEGER | FK → users.id | No | — |
| event_name | VARCHAR(180) | — | No | — |
| category_id | INTEGER | FK → event_categories.id | Yes | — |
| venue_id | INTEGER | FK → event_venues.id | Yes | — |
| event_type | VARCHAR(120) | — | No | — |
| description | TEXT | — | No | — |
| proposed_date | DATE | — | No | — |
| proposed_start_time | TIME | — | Yes | — |
| proposed_end_time | TIME | — | Yes | — |
| proposed_location | VARCHAR(255) | — | No | — |
| expected_attendees | INTEGER | — | No | — |
| supporting_document_path | VARCHAR(255) | — | Yes | — |
| fee_at_submission | NUMERIC(10, 2) | — | No | — |
| status | VARCHAR(30) | — | No | — |
| reviewed_by | INTEGER | FK → users.id | Yes | — |
| decision_date | DATETIME | — | Yes | — |
| remarks | TEXT | — | Yes | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** reference_no.

**Lookup indexes:** ix_event_created (created_at); ix_event_owner_created (requester_id, created_at); ix_event_requests_venue_id (venue_id); ix_event_requests_requester_id (requester_id); ix_event_requests_status (status).

**Relationships:** reviewed_by → users.id (on delete NO ACTION); venue_id → event_venues.id (on delete NO ACTION); category_id → event_categories.id (on delete NO ACTION); requester_id → users.id (on delete NO ACTION).

## A.10 event_reservations

Held/reserved venue intervals, private tokens and release state.

**Created/updated by:** Availability service during submit/accept/decline/expiry/decision.

**Table 25. A.10 event_reservations**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| venue_id | INTEGER | FK → event_venues.id | No | — |
| event_request_id | INTEGER | FK → event_requests.id | Yes | — |
| requester_id | INTEGER | FK → users.id | No | — |
| starts_at | DATETIME | — | No | — |
| ends_at | DATETIME | — | No | — |
| status | VARCHAR(20) | — | No | — |
| hold_token_hash | VARCHAR(64) | — | Yes | — |
| hold_expires_at | DATETIME | — | Yes | — |
| draft_payload | TEXT | — | Yes | — |
| released_at | DATETIME | — | Yes | — |
| release_reason | VARCHAR(255) | — | Yes | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** hold_token_hash; event_request_id.

**Lookup indexes:** ix_event_reservations_status (status); ix_event_reservations_venue_id (venue_id); ix_event_reservation_status_expiry (status, hold_expires_at); ix_event_reservation_venue_interval (venue_id, starts_at, ends_at); ix_event_reservations_requester_id (requester_id).

**Relationships:** requester_id → users.id (on delete NO ACTION); event_request_id → event_requests.id (on delete CASCADE); venue_id → event_venues.id (on delete NO ACTION).

## A.11 event_venues

Configured active venues.

**Created/updated by:** Administrator configuration; reservation lock performs a no-op update.

**Table 26. A.11 event_venues**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| name | VARCHAR(180) | — | No | — |
| address | VARCHAR(255) | — | Yes | — |
| description | TEXT | — | Yes | — |
| is_active | BOOLEAN | — | No | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** name.

**Lookup indexes:** ix_event_venues_is_active (is_active).

## A.12 general_concerns

Resident service concern intake and personnel routing.

**Created/updated by:** Staff/Admin intake and routing; no complete closure route.

**Table 27. A.12 general_concerns**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| reference_no | VARCHAR(40) | — | No | — |
| resident_id | INTEGER | FK → users.id | No | — |
| category | VARCHAR(120) | — | No | — |
| description | TEXT | — | No | — |
| location | VARCHAR(255) | — | Yes | — |
| sensitivity_level | VARCHAR(20) | — | No | — |
| assigned_to | INTEGER | FK → users.id | Yes | — |
| status | VARCHAR(30) | — | No | — |
| encoded_by | INTEGER | FK → users.id | No | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** reference_no.

**Lookup indexes:** ix_general_concerns_status (status); ix_general_concerns_resident_id (resident_id); ix_general_concerns_sensitivity_level (sensitivity_level).

**Relationships:** encoded_by → users.id (on delete NO ACTION); assigned_to → users.id (on delete NO ACTION); resident_id → users.id (on delete NO ACTION).

## A.13 household_members

Resident membership, transfer and join/leave metadata.

**Created/updated by:** Staff/Admin household management.

**Table 28. A.13 household_members**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| household_id | INTEGER | FK → households.id | No | — |
| resident_id | INTEGER | FK → users.id | No | — |
| membership_status | VARCHAR(20) | — | No | — |
| joined_at | DATETIME | — | No | — |
| left_at | DATETIME | — | Yes | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** household_id, resident_id.

**Lookup indexes:** ix_household_members_resident_id (resident_id); ix_household_members_membership_status (membership_status); ix_household_members_household_id (household_id).

**Relationships:** resident_id → users.id (on delete NO ACTION); household_id → households.id (on delete NO ACTION).

## A.14 households

Household number, address, head and status.

**Created/updated by:** Staff/Admin household management.

**Table 29. A.14 households**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| household_number | VARCHAR(80) | — | No | — |
| address | VARCHAR(255) | — | Yes | — |
| head_resident_id | INTEGER | FK → users.id | Yes | — |
| status | VARCHAR(20) | — | No | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** household_number.

**Lookup indexes:** ix_households_status (status).

**Relationships:** head_resident_id → users.id (on delete NO ACTION).

## A.15 notifications

Recipient/channel/content, delivery simulation and read state.

**Created/updated by:** Workflow/authorized sender; recipient read actions.

**Table 30. A.15 notifications**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| user_id | INTEGER | FK → users.id | Yes | — |
| recipient_user_id | INTEGER | FK → users.id | Yes | — |
| recipient_contact | VARCHAR(150) | — | Yes | — |
| title | VARCHAR(180) | — | No | — |
| message | TEXT | — | No | — |
| channel | VARCHAR(20) | — | No | — |
| status | VARCHAR(20) | — | No | — |
| related_type | VARCHAR(30) | — | Yes | — |
| related_id | INTEGER | — | Yes | — |
| sent_at | DATETIME | — | Yes | — |
| read_at | DATETIME | — | Yes | — |
| created_at | DATETIME | — | No | — |

**Unique index keys:** No additional unique index listed beyond the primary key.

**Lookup indexes:** ix_notification_owner_created (user_id, created_at); ix_notifications_channel (channel); ix_notifications_user_id (user_id).

**Relationships:** recipient_user_id → users.id (on delete NO ACTION); user_id → users.id (on delete NO ACTION).

## A.16 password_reset_tokens

Hashed expiring and consumed recovery tokens.

**Created/updated by:** Recovery issue/reset handlers.

**Table 31. A.16 password_reset_tokens**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| user_id | INTEGER | FK → users.id | No | — |
| token_hash | VARCHAR(64) | — | No | — |
| expires_at | DATETIME | — | No | — |
| used_at | DATETIME | — | Yes | — |
| created_at | DATETIME | — | No | — |

**Unique index keys:** token_hash.

**Lookup indexes:** ix_password_reset_tokens_user_id (user_id).

**Relationships:** user_id → users.id (on delete NO ACTION).

## A.17 payment_authorities

Explicit Staff financial permission grants.

**Created/updated by:** Administrator grant/revoke.

**Table 32. A.17 payment_authorities**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| user_id | INTEGER | PK / FK → users.id | No (PK) | — |
| granted_by | INTEGER | FK → users.id | No | — |

**Unique index keys:** No additional unique index listed beyond the primary key.

**Lookup indexes:** No separate lookup index listed.

**Relationships:** granted_by → users.id (on delete NO ACTION); user_id → users.id (on delete NO ACTION).

## A.18 payments

Actual payment or waiver linked to exactly one service request.

**Created/updated by:** Authorized Admin/Staff payment handler; no edit route.

**Table 33. A.18 payments**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| payment_reference | VARCHAR(40) | — | No | — |
| permit_id | INTEGER | FK → permit_applications.id | Yes | — |
| event_id | INTEGER | FK → event_requests.id | Yes | — |
| resident_id | INTEGER | FK → users.id | No | — |
| fee_snapshot | NUMERIC(10, 2) | — | No | — |
| amount_paid | NUMERIC(10, 2) | — | No | — |
| status | VARCHAR(30) | — | No | — |
| payment_method | VARCHAR(60) | — | Yes | — |
| recorded_by | INTEGER | FK → users.id | No | — |
| paid_at | DATETIME | — | No | — |
| notes | TEXT | — | Yes | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Stored CHECK clauses:** (permit_id IS NOT NULL AND event_id IS NULL) OR (event_id IS NOT NULL AND permit_id IS NULL); status IN ('Paid', 'Waived'); fee_snapshot > 0 AND ((status = 'Paid' AND amount_paid = fee_snapshot) OR (status = 'Waived' AND amount_paid = 0)).

**Unique index keys:** event_id; permit_id; payment_reference.

**Lookup indexes:** ix_payments_paid_at (paid_at); ix_payments_resident_id (resident_id).

**Relationships:** recorded_by → users.id (on delete NO ACTION); resident_id → users.id (on delete NO ACTION); event_id → event_requests.id (on delete NO ACTION); permit_id → permit_applications.id (on delete NO

ACTION).

## A.19 permit_applications

Permit ownership, snapshot fee, lifecycle, signature and release data.

**Created/updated by:** Resident/Staff/Admin submission; authorized review/sign/release handlers.

### Table 34. A.19 permit_applications

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| reference_no | VARCHAR(40) | — | No | — |
| applicant_id | INTEGER | FK → users.id | No | — |
| permit_type_id | INTEGER | FK → permit_types.id | No | — |
| purpose | TEXT | — | No | — |
| fee_at_submission | NUMERIC(10, 2) | — | No | — |
| status | VARCHAR(30) | — | No | — |
| application_date | DATETIME | — | Yes | — |
| appointment_date | DATETIME | — | Yes | — |
| decision_date | DATETIME | — | Yes | — |
| decided_by | INTEGER | FK → users.id | Yes | — |
| reviewed_by | INTEGER | FK → users.id | Yes | — |
| remarks | TEXT | — | Yes | — |
| permit_file_path | VARCHAR(255) | — | Yes | — |
| attachment_path | VARCHAR(255) | — | Yes | — |
| request_source | VARCHAR(30) | — | No | — |
| encoded_by_staff_id | INTEGER | FK → users.id | Yes | — |
| office_name | VARCHAR(150) | — | Yes | — |
| endorsed_by | INTEGER | FK → users.id | Yes | — |
| endorsed_at | DATETIME | — | Yes | — |
| endorsement_note | TEXT | — | Yes | — |
| signed_by | INTEGER | FK → users.id | Yes | — |
| signed_at | DATETIME | — | Yes | — |
| signatory_name | VARCHAR(150) | — | Yes | — |
| signatory_title | VARCHAR(150) | — | Yes | — |
| signature_path | VARCHAR(255) | — | Yes | — |
| document_version | INTEGER | — | No | — |
| payment_confirmed_at | DATETIME | — | Yes | — |
| payment_confirmed_by | INTEGER | FK → users.id | Yes | — |
| payment_reference | VARCHAR(120) | — | Yes | — |
| released_by | INTEGER | FK → users.id | Yes | — |
| released_at | DATETIME | — | Yes | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** reference_no.

**Lookup indexes:** ix_permit_created (created_at); ix_permit_owner_date (applicant_id, application_date); ix_permit_applications_released_by (released_by); ix_permit_applications_applicant_id (applicant_id); ix_permit_applications_signed_by (signed_by); ix_permit_applications_status (status); ix_permit_applications_encoded_by_staff_id (encoded_by_staff_id); ix_permit_applications_request_source (request_source); ix_permit_applications_endorsed_by (endorsed_by).

**Relationships:** released_by → users.id (on delete NO ACTION); payment_confirmed_by → users.id (on delete NO ACTION); signed_by → users.id (on delete NO ACTION); endorsed_by → users.id (on delete NO ACTION); encoded_by_staff_id → users.id (on delete NO ACTION); reviewed_by → users.id (on delete NO ACTION); decided_by → users.id (on delete NO ACTION); permit_type_id → permit_types.id (on delete NO ACTION); applicant_id → users.id (on delete NO ACTION).

## A.20 permit_signatories

Per-service authorized Admin and protected signature configuration.

**Created/updated by:** Administrator configuration.

**Table 35. A.20 permit_signatories**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| permit_type_id | INTEGER | FK → permit_types.id | No | — |
| authorized_admin_id | INTEGER | FK → users.id | No | — |
| name | VARCHAR(150) | — | No | — |
| title | VARCHAR(150) | — | No | — |
| file_path | VARCHAR(255) | — | No | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** permit_type_id.

**Lookup indexes:** No separate lookup index listed.

**Relationships:** authorized_admin_id → users.id (on delete NO ACTION); permit_type_id → permit_types.id (on delete NO ACTION).

## A.21 permit_types

Permit/certificate catalog, configured fee and processing information.

**Created/updated by:** Seeder/Administrator configuration.

**Table 36. A.21 permit_types**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| name | VARCHAR(120) | — | No | — |
| description | TEXT | — | Yes | — |
| requirements | TEXT | — | Yes | — |
| processing_time | VARCHAR(180) | — | Yes | — |
| fee | NUMERIC(10, 2) | — | Yes | — |
| fee_is_configured | BOOLEAN | — | No | — |
| validity_days | INTEGER | — | Yes | — |
| is_active | BOOLEAN | — | No | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** name.

**Lookup indexes:** No separate lookup index listed.

## A.22 program_application_history

Previous evidence snapshots and recorded program transitions.

**Created/updated by:** Program review/resubmission handlers.

**Table 37. A.22 program_application_history**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| application_id | INTEGER | FK → program_applications.id | No | — |
| actor_id | INTEGER | FK → users.id | No | — |
| status | VARCHAR(30) | — | No | — |
| requirements_json | TEXT | — | No | — |
| note | TEXT | — | Yes | — |
| created_at | DATETIME | — | No | — |

**Unique index keys:** No additional unique index listed beyond the primary key.

**Lookup indexes:** ix_program_application_history_application_id (application_id).

**Relationships:** actor_id → users.id (on delete NO ACTION); application_id → program_applications.id (on delete NO ACTION).

## A.23 program_applications

Applicant/household claim, snapshot evidence and lifecycle.

**Created/updated by:** Owner or assisted submission; authorized review/Admin completion.

**Table 38. A.23 program_applications**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| reference_no | VARCHAR(40) | — | No | — |
| program_id | INTEGER | FK → community_programs.id | No | — |
| resident_id | INTEGER | FK → users.id | No | — |
| household_id | INTEGER | FK → households.id | Yes | — |
| claim_key | VARCHAR(80) | — | Yes | — |
| status | VARCHAR(30) | — | No | — |
| requirements_json | TEXT | — | No | — |
| encoded_by | INTEGER | FK → users.id | No | — |
| reviewed_by | INTEGER | FK → users.id | Yes | — |
| reviewed_at | DATETIME | — | Yes | — |
| released_by | INTEGER | FK → users.id | Yes | — |
| released_at | DATETIME | — | Yes | — |
| review_note | TEXT | — | Yes | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** reference_no; program_id, claim_key.

**Lookup indexes:** ix_program_application_created (program_id, created_at); ix_program_applications_status (status); ix_program_applications_program_id (program_id); ix_program_applications_resident_id (resident_id).

**Relationships:** released_by → users.id (on delete NO ACTION); reviewed_by → users.id (on delete NO ACTION); encoded_by → users.id (on delete NO ACTION); household_id → households.id (on delete NO ACTION); resident_id → users.id (on delete NO ACTION); program_id → community_programs.id (on delete NO ACTION).

## A.24 queue_entries

Daily assisted permit tickets and stage timestamps.

**Created/updated by:** Front desk intake; Staff/Admin queue actions.

**Table 39. A.24 queue_entries**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| queue_date | DATE | — | No | — |
| queue_number | VARCHAR(30) | — | No | — |
| category | VARCHAR(80) | — | No | — |
| resident_id | INTEGER | FK → users.id | No | — |
| permit_application_id | INTEGER | FK → permit_applications.id | Yes | — |
| status | VARCHAR(20) | — | No | — |
| priority_category | VARCHAR(80) | — | Yes | — |
| called_at | DATETIME | — | Yes | — |
| served_at | DATETIME | — | Yes | — |
| completed_at | DATETIME | — | Yes | — |
| created_by | INTEGER | FK → users.id | No | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** queue_date, queue_number.

**Lookup indexes:** ix_queue_entries_resident_id (resident_id); ix_queue_entries_queue_date (queue_date); ix_queue_entries_permit_application_id (permit_application_id); ix_queue_entries_status (status).

**Relationships:** created_by → users.id (on delete NO ACTION); permit_application_id → permit_applications.id (on delete NO ACTION); resident_id → users.id (on delete NO ACTION).

## A.25 reports

Generated report type/format/filter metadata.

**Created/updated by:** Administrator DOCX export.

**Table 40. A.25 reports**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| generated_by | INTEGER | FK → users.id | No | — |
| report_type | VARCHAR(50) | — | No | — |
| format | VARCHAR(10) | — | No | — |
| filters_json | TEXT | — | Yes | — |
| created_at | DATETIME | — | No | — |

**Unique index keys:** No additional unique index listed beyond the primary key.

**Lookup indexes:** No separate lookup index listed.

**Relationships:** generated_by → users.id (on delete NO ACTION).

## A.26 request_assignments

Permit assignment history and current Active assignment.

**Created/updated by:** Staff/Admin assignment route.

**Table 41. A.26 request_assignments**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| permit_application_id | INTEGER | FK → permit_applications.id | No | — |
| assigned_to_user_id | INTEGER | FK → users.id | No | — |
| assigned_by | INTEGER | FK → users.id | No | — |
| status | VARCHAR(20) | — | No | — |
| note | TEXT | — | Yes | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** No additional unique index listed beyond the primary key.

**Lookup indexes:** ix_request_assignments_status (status); ix_request_assignments_permit_application_id (permit_application_id); ix_request_assignments_assigned_to_user_id (assigned_to_user_id).

**Relationships:** assigned_by → users.id (on delete NO ACTION); assigned_to_user_id → users.id (on delete NO ACTION); permit_application_id → permit_applications.id (on delete NO ACTION).

## A.27 requirement_history

Evidence/review history for individual permit requirements.

**Created/updated by:** Snapshot/resubmission/review handlers; no ordinary history editor.

**Table 42. A.27 requirement_history**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| requirement_id | INTEGER | FK → application_requirements.id | No | — |
| actor_id | INTEGER | FK → users.id | No | — |
| actor_role | VARCHAR(20) | — | No | — |
| status | VARCHAR(30) | — | No | — |
| note | TEXT | — | Yes | — |
| file_path | VARCHAR(255) | — | Yes | — |
| value | TEXT | — | Yes | — |
| created_at | DATETIME | — | No | — |

**Unique index keys:** No additional unique index listed beyond the primary key.

**Lookup indexes:** ix_requirement_history_requirement_id (requirement_id).

**Relationships:** actor_id → users.id (on delete NO ACTION); requirement_id → application_requirements.id (on delete NO ACTION).

## A.28 resident_feedback

One rating/comment per resident and completed permit.

**Created/updated by:** Eligible owner submission; aggregate report reads.

**Table 43. A.28 resident_feedback**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| resident_id | INTEGER | FK → users.id | No | — |
| permit_application_id | INTEGER | FK → permit_applications.id | No | — |
| rating | INTEGER | — | No | — |
| comment | TEXT | — | Yes | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** resident_id, permit_application_id.

**Lookup indexes:** ix_resident_feedback_resident_id (resident_id); ix_resident_feedback_permit_application_id (permit_application_id).

**Relationships:** permit_application_id → permit_applications.id (on delete NO ACTION); resident_id → users.id (on delete NO ACTION).

## A.29 resident_profiles

Residence, ID/address review, consent and account-approval metadata.

**Created/updated by:** Registration/Admin/migration; owner and Administrator review.

### Table 44. A.29 resident_profiles

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| user_id | INTEGER | FK → users.id | No | — |
| date_of_birth | DATE | — | Yes | — |
| sex | VARCHAR(40) | — | Yes | — |
| civil_status | VARCHAR(40) | — | Yes | — |
| house_number_street | VARCHAR(180) | — | Yes | — |
| purok_zone | VARCHAR(120) | — | Yes | — |
| resident_type_id | INTEGER | FK → resident_types.id | Yes | — |
| resident_type_other | VARCHAR(150) | — | Yes | — |
| residency_status | VARCHAR(40) | — | Yes | — |
| years_of_residency | INTEGER | — | Yes | — |
| occupation | VARCHAR(150) | — | Yes | — |
| is_household_head | BOOLEAN | — | No | — |
| household_number | VARCHAR(80) | — | Yes | — |
| emergency_contact_name | VARCHAR(150) | — | Yes | — |
| emergency_contact_relationship | VARCHAR(80) | — | Yes | — |
| emergency_contact_number | VARCHAR(20) | — | Yes | — |
| accepted_id_type_id | INTEGER | FK → accepted_id_types.id | Yes | — |
| valid_id_number | VARCHAR(120) | — | Yes | — |
| valid_id_type_other | VARCHAR(150) | — | Yes | — |
| valid_id_file_path | VARCHAR(255) | — | Yes | — |
| id_verification_status | VARCHAR(30) | — | No | — |
| id_verified_by | INTEGER | FK → users.id | Yes | — |
| id_verified_at | DATETIME | — | Yes | — |
| id_rejection_reason | TEXT | — | Yes | — |
| address_verification_status | VARCHAR(30) | — | No | — |
| address_verified_by | INTEGER | FK → users.id | Yes | — |
| address_verified_at | DATETIME | — | Yes | — |
| address_rejection_reason | TEXT | — | Yes | — |
| privacy_consent | BOOLEAN | — | No | — |
| privacy_consented_at | DATETIME | — | Yes | — |
| approval_status | VARCHAR(20) | — | No | — |
| approval_remarks | TEXT | — | Yes | — |
| reviewed_by | INTEGER | FK → users.id | Yes | — |

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| reviewed_at | DATETIME | — | Yes | — |
| approval_notice_seen_at | DATETIME | — | Yes | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** user_id.

**Lookup indexes:** ix_resident_profiles_accepted_id_type_id (accepted_id_type_id); ix_resident_profiles_approval_status (approval_status); ix_resident_profiles_residency_status (residency_status); ix_resident_profiles_approval_created (approval_status, created_at); ix_resident_profiles_id_verification_status (id_verification_status); ix_resident_profiles_address_verified_by (address_verified_by); ix_resident_profiles_reviewed_by (reviewed_by); ix_resident_profiles_address_verification_status (address_verification_status); ix_resident_profiles_resident_type_id (resident_type_id); ix_resident_profiles_id_verified_by (id_verified_by).

**Relationships:** reviewed_by → users.id (on delete NO ACTION); address_verified_by → users.id (on delete NO ACTION); id_verified_by → users.id (on delete NO ACTION); accepted_id_type_id → accepted_id_types.id (on delete NO ACTION); resident_type_id → resident_types.id (on delete NO ACTION); user_id → users.id (on delete CASCADE).

## A.30 resident_types

Configurable housing relationship catalog.

**Created/updated by:** Migration/Administrator configuration.

**Table 45. A.30 resident_types**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| name | VARCHAR(120) | — | No | — |
| description | TEXT | — | Yes | — |
| is_active | BOOLEAN | — | No | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** name.

**Lookup indexes:** ix_resident_types_is_active (is_active).

## A.31 resident_update_requests

Old/proposed field values and update decision.

**Created/updated by:** Staff/Admin proposal; Administrator approval/rejection.

**Table 46. A.31 resident_update_requests**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| resident_id | INTEGER | FK → users.id | No | — |
| field_name | VARCHAR(80) | — | No | — |
| old_value | TEXT | — | Yes | — |
| proposed_value | TEXT | — | No | — |
| status | VARCHAR(20) | — | No | — |
| reviewed_by | INTEGER | FK → users.id | Yes | — |
| reviewed_at | DATETIME | — | Yes | — |
| created_by | INTEGER | FK → users.id | No | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** No additional unique index listed beyond the primary key.

**Lookup indexes:** ix_resident_update_requests_resident_id (resident_id); ix_resident_update_requests_status (status).

**Relationships:** created_by → users.id (on delete NO ACTION); reviewed_by → users.id (on delete NO ACTION); resident_id → users.id (on delete NO ACTION).

## A.32 schedules

Transaction-linked appointments and processing states.

**Created/updated by:** Owner or operator creation; Staff/Admin status updates.

**Table 47. A.32 schedules**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| reference_no | VARCHAR(40) | — | No | — |
| related_type | VARCHAR(20) | — | No | — |
| related_id | INTEGER | — | No | — |
| requested_by | INTEGER | FK → users.id | No | — |
| scheduled_datetime | DATETIME | — | No | — |
| purpose | VARCHAR(255) | — | No | — |
| note | TEXT | — | Yes | — |
| status | VARCHAR(30) | — | No | — |
| confirmed_by | INTEGER | FK → users.id | Yes | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** reference_no.

**Lookup indexes:** ix_schedules_status (status); ix_schedules_related_type (related_type); ix_schedules_scheduled_datetime (scheduled_datetime); ix_schedules_related_id (related_id).

**Relationships:** confirmed_by → users.id (on delete NO ACTION); requested_by → users.id (on delete NO ACTION).

## A.33 service_requirements

Future permit requirement definitions and order.

**Created/updated by:** Administrator configuration.

**Table 48. A.33 service_requirements**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| permit_type_id | INTEGER | FK → permit_types.id | No | — |
| name | VARCHAR(180) | — | No | — |
| requirement_type | VARCHAR(30) | — | No | — |
| instructions | TEXT | — | Yes | — |
| is_required | BOOLEAN | — | No | — |
| is_active | BOOLEAN | — | No | — |
| position | INTEGER | — | No | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** permit_type_id, name.

**Lookup indexes:** ix_service_requirements_permit_type_id (permit_type_id); ix_service_requirements_is_active (is_active).

**Relationships:** permit_type_id → permit_types.id (on delete NO ACTION).

## A.34 system_settings

Key/value operational settings.

**Created/updated by:** Administrator configuration/migration.

**Table 49. A.34 system_settings**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| key | VARCHAR(100) | — | No | — |
| value | TEXT | — | Yes | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** key.

**Lookup indexes:** No separate lookup index listed.

## A.35 users

Accounts, password hashes, roles and contact data.

**Created/updated by:** Registration/Admin creation; owner profile and Admin account management.

**Table 50. A.35 users**

| Field | SQLite type | Key / reference | Nullable | SQL default |
|---|---|---|---|---|
| id | INTEGER | PK | No (PK) | — |
| username | VARCHAR(80) | — | No | — |
| password_hash | VARCHAR(255) | — | No | — |
| role | VARCHAR(20) | — | No | — |
| full_name | VARCHAR(150) | — | No | — |
| contact_number | VARCHAR(20) | — | Yes | — |
| email | VARCHAR(150) | — | Yes | — |
| address | VARCHAR(255) | — | Yes | — |
| is_active | BOOLEAN | — | No | — |
| created_at | DATETIME | — | No | — |
| updated_at | DATETIME | — | No | — |

**Unique index keys:** email; username.

**Lookup indexes:** ix_users_role (role).

# Appendix B — Route Inventory

This is the full source-declared route inventory; methods shown omit Flask's automatic HEAD/OPTIONS handling. Access summaries combine declared decorators with route checks; Chapter 2 and Chapter 4 explain workflow-specific restrictions. Source locations identify the inspected handler. Paths are internal route patterns, not links to an exposed production service.

### Table 51. Appendix B — Route Inventory

| Method | Actual route | Role / boundary | Purpose / code |
|---|---|---|---|
| GET | / | Public | Index; index at app.py:421 |
| GET | /acknowledgments/<kind>/<int:record_id>.docx | Authenticated; Resident records scoped to owner | Generate acknowledgment DOCX (legacy handler name); submission_receipt at app.py:2168 |
| GET | /activity-logs | Administrator | Activity logs; activity_logs at app.py:2863 |
| GET, POST | /announcements | All authorized view; Staff draft; Admin manages | Announcements; announcements at app.py:976 |
| POST | /announcements/<int:record_id>/approve | Administrator | Approve announcement; approve_announcement at app.py:1017 |
| POST | /announcements/<int:record_id>/delete | Administrator | Archive announcement, preserving row; announcement_delete at app.py:1034 |
| GET, POST | /blotters | Authenticated; Resident records scoped to owner | Blotters; blotters at app.py:2072 |
| GET, POST | /blotters/<int:record_id> | Authenticated; Resident records scoped to owner | Blotter detail; blotter_detail at app.py:2134 |
| GET, POST | /configuration | Administrator | Configuration; configuration at app.py:2880 |
| GET | /dashboard | Authenticated; Resident records scoped to owner | Dashboard; dashboard at app.py:1422 |
| GET, POST | /events | Authenticated; Resident records scoped to owner | Events; events at app.py:1840 |
| GET, POST | /events/<int:record_id> | Authenticated; Resident records scoped to owner | Event detail; event_detail at app.py:1964 |
| POST | /events/recommendations/<token>/accept | Owner of private hold | Accept event recommendation; accept_event_recommendation at app.py:1981 |
| POST | /events/recommendations/<token>/decline | Owner of private hold | Decline event recommendation; decline_event_recommendation at app.py:2055 |
| GET | /favicon.ico | Public | Favicon; favicon at app.py:426 |
| GET, POST | /forgot-password | Public | Forgot password; forgot_password at app.py:720 |
| GET, POST | /front-desk | Staff/Admin operations | Front desk; front_desk at app.py:1085 |
| POST | /front-desk/concerns | Staff/Admin operations | Create general concern; create_general_concern at app.py:1243 |
| POST | /front-desk/concerns/<int:record_id>/route | Staff/Admin operations | Route general concern; route_general_concern at app.py:1266 |
| POST | /front-desk/households | Staff/Admin operations | Manage household; manage_household at app.py:1341 |

| Method | Actual route | Role / boundary | Purpose / code |
|---|---|---|---|
| GET | /front-desk/permits/<int:record_id>/acknowledgment | Authenticated; Resident records scoped to owner | Walkin acknowledgment; walkin_acknowledgment at app.py:1160 |
| POST | /front-desk/permits/<int:record_id>/assign | Staff/Admin operations | Assign permit; assign_permit at app.py:1223 |
| GET | /front-desk/permits/<int:record_id>/claim-stub.docx | Authenticated; Resident records scoped to owner | Generate acknowledgment DOCX (legacy handler name); walkin_claim_stub at app.py:1142 |
| POST | /front-desk/permits/<int:record_id>/release | Staff/Admin operations | Release walkin permit; release_walkin_permit at app.py:1198 |
| POST | /front-desk/queue/<int:queue_id>/<action> | Staff/Admin operations | Update queue; update_queue at app.py:1171 |
| POST | /front-desk/referrals | Staff/Admin operations | Create assistance referral; create_assistance_referral at app.py:1379 |
| POST | /front-desk/resident-updates | Staff/Admin operations | Create resident update request; create_resident_update_request at app.py:1282 |
| GET | /healthz | Public | Healthz; healthz at app.py:431 |
| GET, POST | /login | Public | Login; login at app.py:442 |
| POST | /logout | Authenticated; Resident records scoped to owner | Logout; logout at app.py:498 |
| GET | /notifications | Authenticated; Resident records scoped to owner | Notifications; notifications at app.py:2469 |
| POST | /notifications/<int:record_id>/read | Recipient only | Notification read; notification_read at app.py:2506 |
| POST | /notifications/read-all | Current recipient only | Notifications read all; notifications_read_all at app.py:2517 |
| POST | /notifications/send | Staff/Admin operations | Notification send; notification_send at app.py:2483 |
| GET, POST | /payments | Own financial view; Admin/authorized Staff records, except own | Payments; payments at revision_routes.py:48 |
| GET | /payments/<int:record_id>/receipt.docx | Owner or payment-authorized operator; Paid only | Payment receipt; payment_receipt at revision_routes.py:128 |
| POST | /payments/authority | Administrator | Payment authority; payment_authority at revision_routes.py:111 |
| GET, POST | /permits | Authenticated; Resident records scoped to owner | Permits; permits at app.py:1617 |
| GET, POST | /permits/<int:record_id> | Authenticated; Resident records scoped to owner | Permit detail; permit_detail at app.py:1689 |
| GET | /permits/<int:record_id>/approved-document.docx | Authenticated; Resident records scoped to owner | Approved permit document; approved_permit_document at app.py:2211 |

| Method | Actual route | Role / boundary | Purpose / code |
|---|---|---|---|
| POST | /permits/<int:record_id>/feedback | Owner of Completed permit | Submit permit feedback; submit_permit_feedback at app.py:1400 |
| GET | /permits/<int:record_id>/requirements/<int:requirement_id>/file | Non-owner Staff processor or Administrator | Permit requirement file; permit_requirement_file at app.py:1707 |
| POST | /permits/<int:record_id>/requirements/<int:requirement_id>/review | Staff/Admin operations | Review permit requirement; review_permit_requirement at app.py:1759 |
| POST | /permits/<int:record_id>/requirements/<int:requirement_id>/submit | Eligible owner only | Resubmit permit requirement; resubmit_permit_requirement at app.py:1717 |
| POST | /permits/<int:record_id>/sign | Designated Administrator for service | Sign approved permit; sign_approved_permit at app.py:2251 |
| GET | /privacy-notice | Public | Privacy notice; privacy_notice at app.py:689 |
| GET, POST | /profile | Authenticated; Resident records scoped to owner | Profile; profile at app.py:796 |
| GET, POST | /program-applications/<int:record_id> | Owner view/resubmit; non-owner operators review; Admin final steps | Program application; program_application at revision_routes.py:283 |
| GET | /program-applications/<int:record_id>/acknowledgment.docx | Owner or operational Staff/Admin | Program acknowledgment; program_acknowledgment at revision_routes.py:350 |
| GET | /program-applications/<int:record_id>/files/<int:index> | Owner or operational Staff/Admin | Program file; program_file at revision_routes.py:373 |
| GET, POST | /programs | All approved users view; Staff/Admin draft | Programs; programs at revision_routes.py:148 |
| GET, POST | /programs/<int:record_id> | Resident own application; Staff/Admin assist | Program detail; program_detail at revision_routes.py:235 |
| POST | /programs/<int:record_id>/close | Administrator | Close program; close_program at revision_routes.py:221 |
| POST | /programs/<int:record_id>/publish | Administrator | Publish program; publish_program at revision_routes.py:202 |
| POST | /programs/<int:record_id>/requirements | Administrator | Program requirements; program_requirements at revision_routes.py:186 |
| GET, POST | /register | Public | Register; register at app.py:506 |
| GET | /reports | Administrator | Reports; reports at app.py:2792 |
| GET | /reports/chart/<int:index>.png | Administrator | Report chart; report_chart at app.py:2843 |
| POST | /reports/export/<fmt> | Administrator | Report export; report_export at app.py:2809 |
| GET, POST | /reset-password/<token> | Public | Reset password; reset_password at app.py:757 |

| Method | Actual route | Role / boundary | Purpose / code |
|---|---|---|---|
| GET | /resident-identifications/<int:user_id>/document | Owner or Administrator | Resident identification document; resident_identification_document at app.py:695 |
| POST | /resident-updates/<int:record_id>/review | Administrator | Review resident update; review_resident_update at app.py:1306 |
| GET, POST | /schedules | Authenticated; Resident records scoped to owner | Schedules; schedules at app.py:2344 |
| POST | /schedules/<int:record_id>/status | Staff/Admin allowed transitions; owner cannot act as Resident | Schedule status; schedule_status at app.py:2440 |
| GET | /status | Authenticated; Resident records scoped to owner | Transaction status; transaction_status at app.py:1573 |
| GET | /submissions/<kind>/<int:record_id>/success | Authenticated; Resident records scoped to owner | Submission success; submission_success at app.py:2149 |
| GET, POST | /users | Staff read-only; Administrator manages | Users; users at app.py:2528 |
| GET | /users/<int:record_id> | Administrator | User review; user_review at app.py:2583 |
| POST | /users/<int:record_id>/approval | Administrator | User approval; user_approval at app.py:2677 |
| POST | /users/<int:record_id>/role | Administrator | User role; user_role at app.py:2747 |
| POST | /users/<int:record_id>/status | Administrator | User account status; user_account_status at app.py:2774 |
| POST | /users/<int:record_id>/verification/<kind> | Administrator | User verification; user_verification at app.py:2608 |

# Appendix C — Source Evidence and Reproduction

Source references in this document point to the local project files. No external papers or survey references are invented. The companion `source_evidence.json` records paths, line counts, SHA-256 source fingerprints, symbols, routes, tests and schema metadata. The source review began October 5, 2026; document completion is October 7, 2026 (Asia/Manila). The older browser/performance evidence and current captured acceptance run are distinguished.

### Table 52. Appendix C — Source Evidence and Reproduction

| Inspected source | Lines | Purpose |
|---|---|---|
| barangay_system/app.py | 3139 | Application/service/data logic |
| barangay_system/backup_sqlite.py | 41 | Application/service/data logic |
| barangay_system/config.py | 63 | Application/service/data logic |
| barangay_system/migration.py | 369 | Application/service/data logic |
| barangay_system/models.py | 851 | Application/service/data logic |
| barangay_system/permissions.py | 76 | Application/service/data logic |
| barangay_system/revision_routes.py | 382 | Application/service/data logic |
| barangay_system/seed.py | 22 | Application/service/data logic |
| barangay_system/services/__init__.py | 0 | Application/service/data logic |
| barangay_system/services/analytics.py | 270 | Application/service/data logic |
| barangay_system/services/documentgen.py | 454 | Application/service/data logic |
| barangay_system/services/event_availability.py | 207 | Application/service/data logic |
| barangay_system/services/notify.py | 25 | Application/service/data logic |
| barangay_system/services/revision.py | 91 | Application/service/data logic |
| barangay_system/services/uploads.py | 27 | Application/service/data logic |
| barangay_system/static/css/style.css | 17 | Browser presentation |
| barangay_system/static/js/charts.js | 46 | Browser presentation |
| barangay_system/templates/_icons.html | 1 | Template/presentation |
| barangay_system/templates/_pagination.html | 1 | Template/presentation |
| barangay_system/templates/_program_requirements.html | 1 | Template/presentation |
| barangay_system/templates/activity_logs.html | 1 | Template/presentation |
| barangay_system/templates/announcements.html | 11 | Template/presentation |
| barangay_system/templates/auth_base.html | 1 | Template/presentation |
| barangay_system/templates/base.html | 1 | Template/presentation |

| Inspected source | Lines | Purpose |
|---|---|---|
| barangay_system/templates/configuration.html | 210 | Template/presentation |
| barangay_system/templates/dashboard.html | 1 | Template/presentation |
| barangay_system/templates/error.html | 1 | Template/presentation |
| barangay_system/templates/event_recommendation.html | 6 | Template/presentation |
| barangay_system/templates/forgot_password.html | 1 | Template/presentation |
| barangay_system/templates/front_desk.html | 14 | Template/presentation |
| barangay_system/templates/login.html | 1 | Template/presentation |
| barangay_system/templates/notifications.html | 1 | Template/presentation |
| barangay_system/templates/payments.html | 5 | Template/presentation |
| barangay_system/templates/privacy_notice.html | 1 | Template/presentation |
| barangay_system/templates/profile.html | 7 | Template/presentation |
| barangay_system/templates/program_application.html | 5 | Template/presentation |
| barangay_system/templates/program_detail.html | 7 | Template/presentation |
| barangay_system/templates/programs.html | 10 | Template/presentation |
| barangay_system/templates/record_detail.html | 9 | Template/presentation |
| barangay_system/templates/records.html | 17 | Template/presentation |
| barangay_system/templates/register.html | 32 | Template/presentation |
| barangay_system/templates/reports.html | 6 | Template/presentation |
| barangay_system/templates/reset_password.html | 1 | Template/presentation |
| barangay_system/templates/schedules.html | 12 | Template/presentation |
| barangay_system/templates/submission_success.html | 1 | Template/presentation |
| barangay_system/templates/transaction_status.html | 1 | Template/presentation |
| barangay_system/templates/user_review.html | 71 | Template/presentation |

| Inspected source | Lines | Purpose |
|---|---|---|
| barangay_system/templates/users.html | 9 | Template/presentation |
| barangay_system/templates/walkin_acknowledgment.html | 6 | Template/presentation |
| barangay_system/tests/audit_inventory.py | 64 | Test/audit evidence |
| barangay_system/tests/browser_audit.py | 100 | Test/audit evidence |
| barangay_system/tests/browser_qa_accounts.py | 41 | Test/audit evidence |
| barangay_system/tests/read_only_audit.py | 58 | Test/audit evidence |
| barangay_system/tests/sqlite_restart_probe.py | 32 | Test/audit evidence |
| barangay_system/tests/test_revision.py | 385 | Test/audit evidence |
| barangay_system/tests/test_stabilization.py | 199 | Test/audit evidence |
| barangay_system/tests/test_system.py | 993 | Test/audit evidence |
| barangay_system/tests/validate_sqlite.py | 46 | Test/audit evidence |
| barangay_system/workflows.py | 37 | Application/service/data logic |

# Appendix D — Final Documentation Verification

- Final source modules, route registrations, workflows, templates, static files, configuration, schema, migrations and audit evidence were inspected.

- SQLite is documented as the current database; old MariaDB material is classified as historical.

- All three roles, implemented modules and direct/route-specific permission boundaries are covered.

- Registration, ID/address verification, dynamic requirements, permits, events, blotters, walk-in, queues, payments, households, programs and announcement expiry are covered, including partial behavior.

- Reports, analytics, DOCX, audit logs, security, database fields, ERDs, code examples and actual test evidence are included.

- Resident, Staff and Administrator manuals, maintenance/backup instructions, glossary and at least 75 defense questions are included.

- The separate condensed cheat sheet covers purpose, roles, workflows, technologies, database, controls and limits in three pages.

- Screenshots use synthetic test fixtures. Operational rows, private ID files, signatures, passwords and secrets are excluded.

- Editable Word text/tables and PDF structure are checked by the documentation validation script; actual Microsoft Word application rendering and physical print quality remain unverified.